CHAINSENSE2

MANUALE TECNICO E DOCUMENTAZIONE SOFTWARE

Sistema di acquisizione, archiviazione e monitoraggio di dati energetici



Versione documento: 1
Firmware ESP32: 1.0
Ambiente: ESP32, C++, Python, SQLite, Streamlit, Docker
Autore: Mattia Floriani
Contesto: Progetto di tirocinio curriculare presso Cinetix Group
Periodo: Ottobre–novembre 2026

1. Introduzione **2**
2. Architettura generale del sistema **4**
3. Architettura hardware e collegamenti **5**
4 Architettura e funzionamento del firmware ESP32 ..... **10**
5. Protocollo CCP e decodifica delle Compact Frame **22**
6. Comunicazione ESP32–Python e acquisizione dati **34**
7. Database SQLite e persistenza delle misurazioni **43**
8. Dashboard web Streamlit **49**
9. Docker e containerizzazione **58**
10. Installazione, configurazione e avvio del sistema **70**
11. Diagnostica, gestione degli errori e troubleshooting **74**
12. Limiti, sviluppi futuri e conclusioni **79**

1. INTRODUZIONE

1.1 PRESENTAZIONE DEL PROGETTO

ChainSense2 è un sistema prototipale per l'acquisizione, l'elaborazione, l'archiviazione e la visualizzazione di dati relativi alla potenza elettrica.

Il progetto utilizza un microcontrollore ESP32 collegato a un dispositivo Chain2Gate Engine, attraverso il quale vengono ricevute informazioni provenienti da un contatore elettronico Open Meter.

Il firmware, sviluppato in C++, gestisce la comunicazione seriale, verifica l'integrità dei messaggi ricevuti e interpreta le informazioni di interesse. La potenza istantanea viene visualizzata localmente attraverso un display OLED e trasmessa a un'applicazione Python mediante collegamento seriale USB.

L'applicazione Python provvede all'acquisizione e all'archiviazione delle misurazioni in un database SQLite.

Una dashboard web, realizzata con il framework Streamlit, permette di consultare le ultime misurazioni, analizzare lo storico e visualizzare l'andamento della potenza attraverso grafici e indicatori.

La dashboard viene eseguita all'interno di un container Docker, mentre il processo responsabile della comunicazione seriale e della scrittura sul database rimane operativo sul sistema host Windows.

1.2 OBIETTIVI DEL SISTEMA

L'obiettivo principale di ChainSense2 è realizzare una soluzione modulare per il monitoraggio della potenza elettrica, integrando componenti embedded e applicazioni software.

Le principali funzionalità implementate comprendono:

- Ricezione delle Compact Frame attraverso comunicazione UART.
- Verifica dell'integrità dei messaggi mediante checksum.
- Gestione delle risposte previste dal protocollo CCP.
- Riconoscimento delle Compact Frame CF3 e CF21.
- Estrazione della potenza elettrica istantanea.
- Visualizzazione locale tramite display OLED e LED diagnostici.
- Trasmissione dei dati in formato JSON.
- Archiviazione persistente mediante SQLite.
- Consultazione delle misurazioni attraverso dashboard web.
- Visualizzazione di grafici, statistiche e storico.
- Esportazione delle misurazioni in formato CSV.
- Esecuzione della dashboard in ambiente Docker.

1.3 AMBITO DI APPLICAZIONE

ChainSense2 è stato sviluppato come prototipo dimostrativo durante un tirocinio formativo nell'ambito dello sviluppo software e dei sistemi embedded.

L'architettura consente di integrare diverse tecnologie mantenendo separate le responsabilità di acquisizione, elaborazione, archiviazione e visualizzazione.

2. ARCHITETTURA GENERALE DEL SISTEMA

2.1 ORGANIZZAZIONE DEI COMPONENTI

ChainSense2 adotta un'architettura suddivisa in componenti hardware e software.

Il flusso delle informazioni parte dal contatore Open Meter, prosegue attraverso il modulo Chain2Gate Engine e raggiunge il microcontrollore ESP32.

Dopo l'elaborazione, il microcontrollore trasmette i dati al computer tramite USB seriale, utilizzando il formato JSON.

L'applicazione Python riceve le misurazioni e le memorizza in SQLite, rendendole disponibili alla dashboard Streamlit.

2.2 SUDDIVISIONE DELLE RESPONSABILITÀ

Livello embedded — ESP32

Il firmware gestisce la comunicazione con Chain2Gate Engine, la verifica e l'interpretazione delle trame, il display OLED e gli indicatori di stato.

Livello di acquisizione — Python

Il modulo `serial_reader.py` legge i dati trasmessi dall'ESP32, interpreta i messaggi JSON e invoca le funzionalità di persistenza.

Livello di archiviazione — SQLite

Il modulo `storage.py` inizializza il database e registra le misurazioni in una tabella dedicata.

Livello applicativo — Streamlit

Il modulo `dashboard.py` interroga il database, elabora le misurazioni tramite Pandas e presenta i risultati nell'interfaccia web.

Ambiente di esecuzione — Docker

Docker permette di eseguire la dashboard in un ambiente Linux containerizzato. Il database rimane sul sistema host e viene reso accessibile al container mediante un bind mount in sola lettura.

2.3 CARATTERISTICHE ARCHITETTURALI

La separazione tra i diversi componenti costituisce una delle principali scelte progettuali.

Ogni modulo svolge una responsabilità specifica, riducendo le dipendenze tra acquisizione, memorizzazione e presentazione dei dati.

Questa impostazione semplifica la manutenzione, permette di verificare separatamente i componenti e facilita eventuali evoluzioni del sistema.

Ad esempio, la dashboard può consultare le misurazioni archiviate senza accedere direttamente alla porta seriale dell'ESP32.

Allo stesso modo, la logica di acquisizione può continuare a operare senza richiedere che l'interfaccia web sia aperta nel browser.

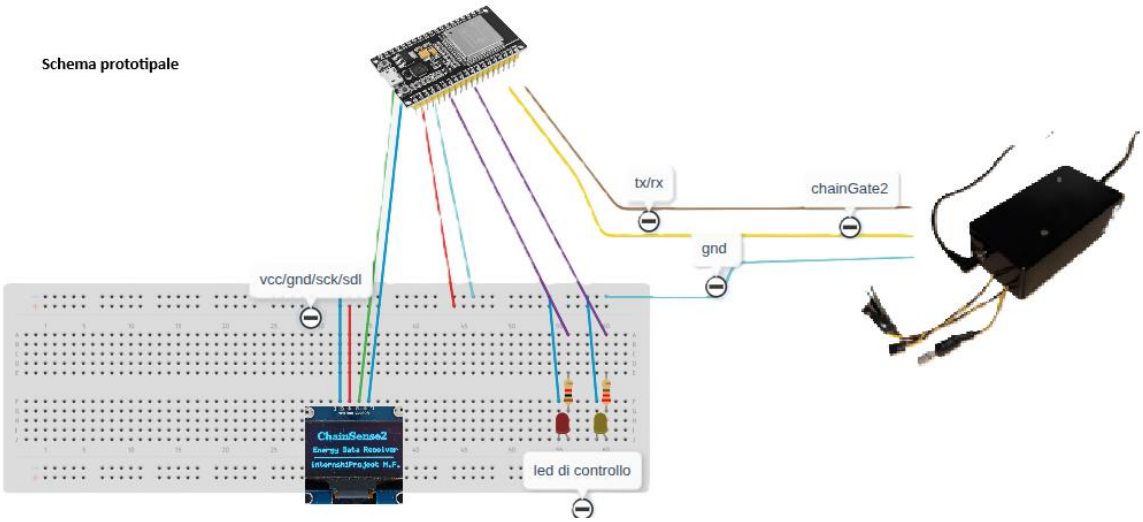
3. ARCHITETTURA HARDWARE E COLLEGAMENTI

3.1 PANORAMICA DELL'HARDWARE

Il sistema ChainSense2 utilizza una piattaforma hardware basata sul microcontrollore ESP32, incaricato di ricevere ed elaborare le informazioni energetiche provenienti dal modulo Chain2Gate Engine.

I dati vengono acquisiti attraverso un'interfaccia seriale UART, mentre le informazioni principali sono visualizzate localmente mediante un display OLED. Due indicatori LED permettono inoltre di monitorare visivamente l'attività del sistema e segnalare eventuali anomalie nella comunicazione.

L'architettura hardware è stata progettata per mantenere separati i collegamenti dedicati alla ricezione dei dati, alla visualizzazione e alla comunicazione con il computer.



3.1.1 COMPONENTI UTILIZZATI

Componente	Modello / Caratteristiche	Funzione
Microcontrollore	ESP32 DOIT DevKit V1	Unità centrale di elaborazione
Interfaccia energetica	Chain2Gate Engine GRID Edition	Ricezione dei dati dal contatore Open Meter
Display	OLED SH1106, 128×64 pixel, I²C	Visualizzazione locale
LED giallo	Indicatore luminoso	Segnalazione della ricezione
LED rosso	Indicatore luminoso	Segnalazione degli errori
Collegamento USB	Porta USB dell'ESP32	Alimentazione, debug e trasmissione dati al PC
		Assemblaggio dei componenti
Breadboard e cablaggi	Collegamenti di prototipazione	

3.2 MICROCONTROLLORE ESP32

L'ESP32 costituisce l'unità centrale del sistema embedded.

Il firmware è sviluppato in linguaggio C++ mediante il framework Arduino, utilizzando Visual Studio Code e l'ambiente PlatformIO.

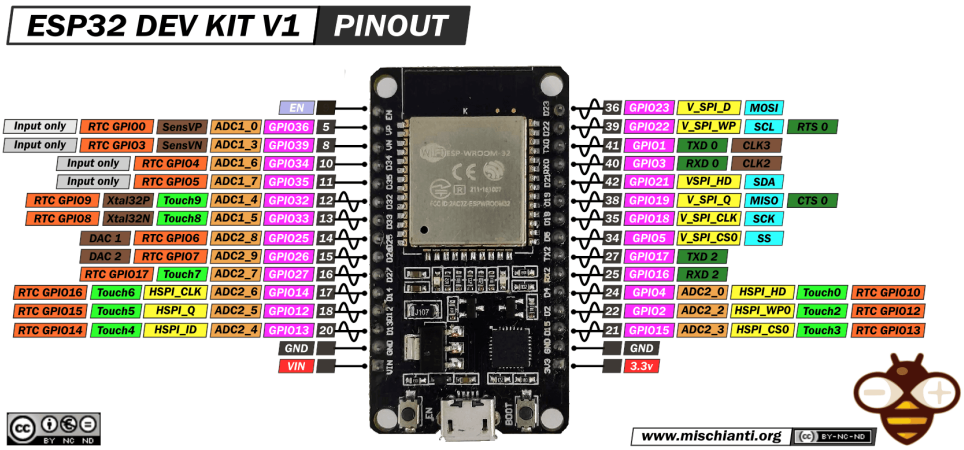
Il microcontrollore svolge le seguenti operazioni:

- Gestione della comunicazione UART con il modulo Chain2Gate Engine.
- Ricezione e memorizzazione temporanea dei messaggi seriali.
- Controllo dell'integrità delle trame mediante checksum.



- Decodifica delle Compact Frame supportate.
- Estrazione della potenza elettrica istantanea.
- Aggiornamento del display OLED e degli indicatori LED.
- Trasmissione dei dati elaborati al computer tramite USB seriale.

Il firmware utilizza una logica di gestione non bloccante, basata su macchine a stati e temporizzazioni tramite `millis()`, per coordinare le diverse attività.



3.2.1 ASSEGNAZIONE DEI PIN

La configurazione dei collegamenti è definita all'interno del firmware mediante costanti.

GPIO ESP32	Periferica	Funzione
GPIO 16	Chain2Gate	RX UART2
GPIO 17	Chain2Gate	TX UART2
GPIO 21	Display OLED	SDA I²C
GPIO 22	Display OLED	SCL I²C
GPIO 14	LED giallo	Indicatore ricezione
GPIO 12	LED rosso	Indicatore errore

L'inizializzazione delle periferiche viene effettuata nella funzione `setup()`.

```
Wire.begin(21, 22);
oled.begin();

Serial.begin(115200);
Serial2.begin(115200, SERIAL_8N1, RXD2, TXD2);
```

La porta `Serial` viene utilizzata per la comunicazione USB con il computer, mentre `Serial2` è dedicata allo scambio di messaggi con Chain2Gate Engine.

3.3 MODULO CHAIN2GATE ENGINE

Il Chain2Gate Engine GRID Edition costituisce l'interfaccia tra il contatore elettronico Open Meter e il microcontrollore ESP32.

Il modulo gestisce la comunicazione con il contatore attraverso la tecnologia **Chain 2** e rende disponibili le informazioni ricevute mediante un protocollo seriale denominato **CCP (Chain2Gate Communication Protocol).**

Questa architettura permette all'ESP32 di acquisire le informazioni energetiche senza implementare direttamente il protocollo di comunicazione power line utilizzato dal contatore.

3.3.1 COLLEGAMENTO SERIALE UART

La comunicazione tra Chain2Gate ed ESP32 utilizza un'interfaccia UART asincrona.

I parametri previsti sono:

Parametro	Valore
Velocità	115200 baud
Bit di dati	8
Parità	Nessuna
Bit di stop	1
Modalità	Full-duplex
Protocollo applicativo	CCP

Le linee di trasmissione e ricezione devono essere collegate in modo incrociato: il segnale TX di un dispositivo deve raggiungere il segnale RX dell'altro.

I dispositivi devono inoltre condividere un riferimento elettrico comune tramite GND.

La compatibilità dei livelli elettrici delle interfacce deve essere verificata prima di effettuare i collegamenti.

Il firmware imposta la comunicazione attraverso la porta UART2 dell'ESP32, lasciando disponibile la connessione USB seriale per il trasferimento dei dati verso il computer.

3.4 DISPLAY OLED SH1106

Il prototipo integra un display OLED 1,3” con risoluzione di 128×64 pixel, basato sul controller SH1106.

La comunicazione con il microcontrollore avviene attraverso il bus I²C, che utilizza due segnali principali:

- **SDA (Serial Data):** linea utilizzata per il trasferimento dei dati.
- **SCL (Serial Clock):** linea di sincronizzazione della comunicazione.

La gestione grafica del display è affidata alla libreria U8g2.

Nel firmware viene utilizzata la seguente configurazione:

```
U8G2_SH1106_128X64_NONAME_F_HW_I2C oled(
```

```
    U8G2_R0,
    U8X8_PIN_NONE
);
```

La configurazione utilizza l'interfaccia I²C hardware dell'ESP32 e una modalità di gestione grafica con buffer completo.

3.4.1 VISUALIZZAZIONE DURANTE L'AVVIO

Durante l'inizializzazione, il display presenta una sequenza di schermate informative.

La sequenza comprende:

1. Presentazione iniziale del sistema.
2. Esecuzione del test degli indicatori LED.
3. Visualizzazione del logo ChainSense2.
4. Passaggio alla schermata di attesa dei dati.



La schermata identificativa contiene il nome del progetto e la descrizione ENERGY DATA RECEIVER.

3.4.2 VISUALIZZAZIONE DURANTE IL FUNZIONAMENTO

Terminata la sequenza iniziale, il display passa alla modalità operativa.

Dopo la ricezione e la decodifica di una Compact Frame riconosciuta, vengono visualizzati:

- Il nome del sistema.
- La potenza elettrica istantanea espressa in watt.
- Il tipo di Compact Frame ricevuta, CF3 oppure CF21.
- L'indicazione `RX OK`.



Il display costituisce quindi un'interfaccia locale di consultazione, indipendente dall'apertura della dashboard web sul computer.

3.5 INDICATORI LED

ChainSense2 utilizza due LED per segnalare il comportamento operativo del microcontrollore.

Il controllo degli indicatori è affidato al firmware, che gestisce accensione e spegnimento attraverso i rispettivi GPIO.

3.5.1 LED GIALLO — RICEZIONE DATI

Il LED giallo è collegato al GPIO 14.

Durante il normale funzionamento viene mantenuto acceso e, in seguito alla ricezione di una notifica valida, si spegne temporaneamente per circa 200 millisecondi.

Questa segnalazione permette di riconoscere visivamente l'attività di ricezione.

3.5.2 LED ROSSO — SEGNALAZIONE ERRORI

Il LED rosso è collegato al GPIO 12.

Il firmware lo attiva quando viene rilevata una condizione di errore gestita, come un timeout di ricezione, un overflow del buffer oppure un errore di checksum su una notifica CCP riconoscibile.

L'indicatore rimane attivo per circa tre secondi, trascorsi i quali viene spento automaticamente.

3.5.3 TEST DEGLI INDICATORI ALL'AVVIO

Durante l'inizializzazione viene eseguita una sequenza di verifica dei LED.

Entrambi gli indicatori vengono accesi e spenti alternativamente per tre cicli, utilizzando un intervallo di 200 millisecondi tra le commutazioni.

Il test consente di verificare visivamente la risposta degli indicatori prima dell'ingresso nello stato operativo.

3.6 ALIMENTAZIONE E COLLEGAMENTO AL COMPUTER

Nel prototipo l'ESP32 viene collegato al computer mediante USB.

Il collegamento permette l'alimentazione della scheda, il caricamento del firmware e la comunicazione seriale con il software Python.

La porta seriale USB viene utilizzata sia per i messaggi diagnostici sia per la trasmissione delle misurazioni strutturate in formato JSON.

Il modulo Chain2Gate dispone di un proprio collegamento di comunicazione verso l'ESP32 attraverso UART.

La gestione dei dati tra le due interfacce seriali rimane separata a livello firmware.

3.7 CONSIDERAZIONI SULL'ARCHITETTURA HARDWARE

La configurazione hardware di ChainSense2 consente di distinguere tre funzioni principali: acquisizione, elaborazione e visualizzazione.

L'utilizzo di UART2 per la comunicazione con Chain2Gate e della porta USB seriale per il collegamento al computer evita di condividere un'unica interfaccia fisica per entrambe le comunicazioni.

Il display OLED e gli indicatori LED forniscono inoltre un riscontro locale sul comportamento del dispositivo, semplificando le attività di verifica e diagnostica.

La realizzazione su breadboard rende possibile modificare i collegamenti e sperimentare diverse configurazioni, mantenendo il sistema adatto alle attività di sviluppo e laboratorio.

Questa soluzione costituisce una piattaforma prototipale sulla quale è possibile sviluppare e verificare le successive componenti software di ChainSense2.

4. ARCHITETTURA E FUNZIONAMENTO DEL FIRMWARE ESP32

4.1 INTRODUZIONE AL FIRMWARE

Il firmware di ChainSense2 costituisce il componente software responsabile dell'acquisizione e dell'elaborazione dei dati provenienti dal modulo Chain2Gate Engine.

Sviluppato in linguaggio C++ mediante il framework Arduino per ESP32, il programma gestisce la comunicazione seriale UART, la verifica dell'integrità delle trame ricevute, l'interpretazione delle Compact Frame e la visualizzazione delle informazioni.

Il firmware si occupa inoltre della trasmissione delle misurazioni verso un'applicazione Python, utilizzando messaggi JSON attraverso la porta seriale USB.

L'organizzazione del codice segue un modello basato su funzioni con responsabilità distinte e macchine a stati, con l'obiettivo di garantire un flusso di esecuzione ordinato e una gestione delle operazioni senza ricorrere a ritardi bloccanti.

Il codice sorgente di riferimento è contenuto nel file `main 1.0.cpp`, identificato come versione firmware 1.0.

4.2 LIBRERIE UTILIZZATE

Il firmware utilizza tre librerie principali, necessarie per la gestione del microcontrollore e delle periferiche.

Libreria	Funzione
`Arduino.h`	Fornisce le funzionalità di base del framework Arduino, comprese le operazioni sui GPIO, la gestione seriale e le temporizzazioni.
`U8g2lib.h`	Gestisce la rappresentazione grafica e la comunicazione con il display OLED SH1106.
`Wire.h`	Fornisce il supporto alla comunicazione I²C utilizzata dal display.

Le librerie vengono importate all'inizio del programma mediante le direttive:

```
#include <Arduino.h>
#include <U8g2lib.h>
#include <Wire.h>
```

4.3 ORGANIZZAZIONE GENERALE DEL PROGRAMMA

Il firmware mantiene la struttura tipica delle applicazioni sviluppate con il framework Arduino, basata sulle funzioni `setup()` e `loop()`.

La funzione `setup()` viene eseguita all'accensione o al riavvio del microcontrollore e inizializza le periferiche necessarie.

La funzione `loop()` viene invece eseguita ripetutamente e coordina le attività del sistema in base allo stato operativo corrente.

Per rendere il codice più comprensibile e manutenibile, le attività principali sono state suddivise in funzioni dedicate.

4.3.1 FUNZIONI PRINCIPALI

Funzione	Responsabilità
`setup()`	Inizializzazione delle periferiche hardware.
`loop()`	Coordinamento generale del programma.
`gestioneAvvio()`	Sequenza iniziale e test delle periferiche di segnalazione.
`gestioneSeriale()`	Richiamo della procedura di ricezione UART.

`riceviDati()`	Ricezione dei byte e verifica delle trame CCP.
`parseFrame()`	Riconoscimento delle Compact Frame ed estrazione della potenza.
`aggiornaSerialeMonitor()`	Visualizzazione delle informazioni diagnostiche.
`inviaDatiPython()`	Trasmissione delle misurazioni in formato JSON.
`aggiornaDisplay()`	Aggiornamento del display OLED.
`aggiornaLed()`	Gestione temporizzata degli indicatori luminosi.
`resetRicezione()`	Ripristino delle variabili di ricezione.

La separazione tra queste funzioni consente di mantenere distinta la logica di acquisizione da quella di elaborazione e presentazione dei dati.

4.4 INIZIALIZZAZIONE DEL SISTEMA — SETUP()

La funzione `setup()` prepara le periferiche utilizzate durante il funzionamento del dispositivo.

Le operazioni principali comprendono:

1. Inizializzazione del bus I²C.
2. Avvio del display OLED.
3. Configurazione della porta seriale USB.
4. Configurazione della porta UART2.
5. Impostazione dei pin dei LED come uscite digitali.

La configurazione utilizzata è la seguente:

```
void setup()
{
    Wire.begin(21, 22);
    oled.begin();

    Serial.begin(115200);
    Serial2.begin(115200, SERIAL_8N1, RXD2, TXD2);

    pinMode(ledRosso, OUTPUT);
    pinMode(ledGiallo, OUTPUT);

    Serial.println("Inizializzazione completata....");
}
```

La presenza di due interfacce seriali distinte permette di separare la comunicazione verso Chain2Gate Engine dalla trasmissione dei dati e dei messaggi diagnostici verso il computer.

4.5 GESTIONE DEL CICLO PRINCIPALE — LOOP()

La funzione `loop()` costituisce il punto centrale di coordinamento del firmware.

Il comportamento del sistema dipende dalla variabile `statoSistema`, definita attraverso un'enumerazione C++.

```
enum class StatoSistema
{
    AVVIO,
    OPERATIVO,
    ERRORE
};
```

Ogni stato rappresenta una diversa condizione operativa.

Stato	Descrizione
`AVVIO`	Esegue la sequenza di inizializzazione e presentazione.
`OPERATIVO`	Gestisce ricezione, elaborazione e visualizzazione dei dati.
`ERRORE`	Stato previsto dall'architettura, senza una procedura dedicata implementata nella versione corrente.

Durante la normale esecuzione, il programma passa dallo stato `AVVIO` allo stato `OPERATIVO`.

Il coordinamento avviene attraverso un'istruzione `switch`, che seleziona il comportamento in funzione dello stato corrente.

4.5.1 SEQUENZA OPERATIVA

Quando il sistema è operativo, vengono eseguite le seguenti attività:

1. Controllo della porta UART e ricezione dei dati disponibili.
2. Verifica della presenza di una trama completa.
3. Identificazione della Compact Frame ed estrazione dei dati.
4. Aggiornamento del monitor seriale.
5. Trasmissione della misurazione in formato JSON, se riconosciuta.
6. Aggiornamento del display OLED.
7. Ripristino della ricezione per il messaggio successivo.
8. Aggiornamento degli indicatori LED.

Il codice che coordina queste attività è:

```
case StatoSistema::OPERATIVO:
    gestioneSeriale();

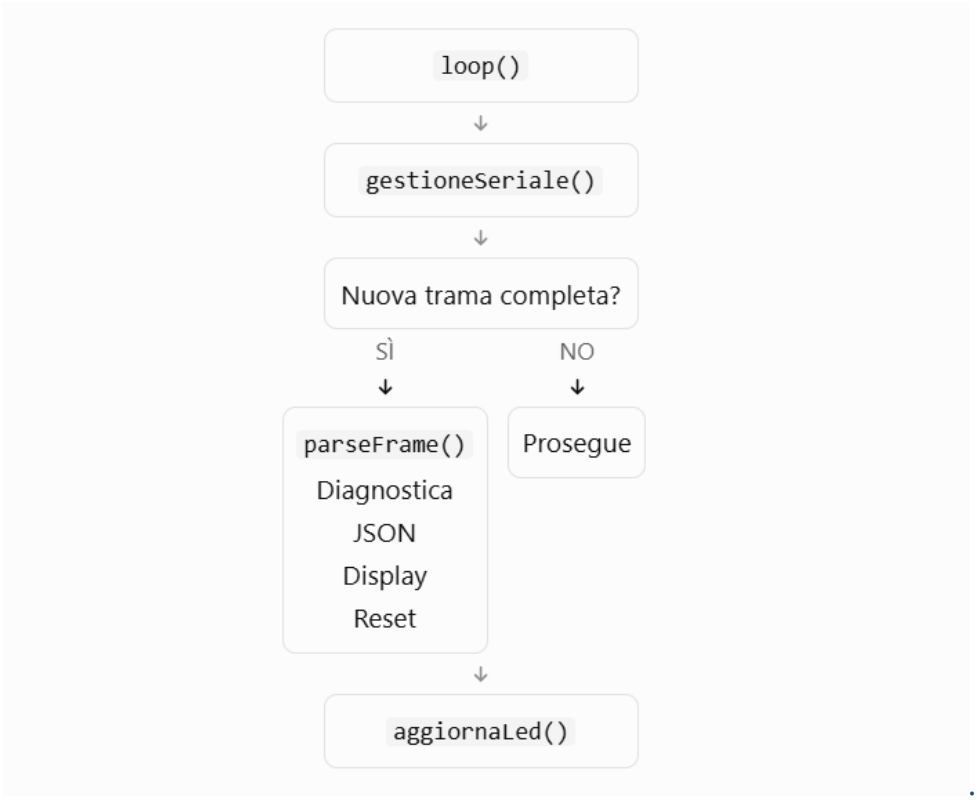
    if (nuoviDatiSeriali)
    {
        parseFrame();
        aggiornaSerialeMonitor();
        inviaDatiPython();
```

```
        aggiornaDisplay();
        resetRicezione();
    }

    aggiornaLed();
    break;
```

La variabile booleana `nuoviDatiSeriali` indica la disponibilità di una nuova notifica completa e validata dal controllo di ricezione.

L'elaborazione avviene soltanto quando questa condizione risulta vera.



4.6 GESTIONE DELLE MACCHINE A STATI

Una delle caratteristiche principali del firmware è l'utilizzo di macchine a stati finite, comunemente indicate come FSM (FINITE STATE MACHINE).

Una macchina a stati definisce un insieme di condizioni operative e le regole che determinano il passaggio da una condizione all'altra.

In ChainSense2 questo modello viene applicato a tre livelli:

- Stato generale del sistema.
- Sequenza di avvio.
- Ricezione dei messaggi seriali.

L'utilizzo di enumerazioni `enum class` permette di rappresentare gli stati mediante identificatori espliciti, migliorando la leggibilità del codice e riducendo l'impiego di valori numerici privi di significato immediato.

4.6.1 MACCHINA A STATI DEL SISTEMA

La macchina a stati principale distingue l'inizializzazione dal normale funzionamento.

All'accensione il sistema entra nello stato `AVVIO`.

Una volta completata la sequenza iniziale, passa allo stato `OPERATIVO`, nel quale rimane durante le attività di monitoraggio.

Lo stato `ERRORE` è previsto nell'enumerazione, ma nella versione documentata non viene utilizzato come destinazione delle anomalie di ricezione. Queste ultime sono gestite attraverso contatori, reset della ricezione e segnalazioni LED.

4.6.2 MACCHINA A STATI DI AVVIO

La sequenza iniziale è gestita attraverso una seconda enumerazione.

```
enum class StatoAvvio
{
    PRESENTAZIONE,
    TEST_LED,
    LOGO,
    COMPLETATO
};
```

Stato	Attività
`PRESENTAZIONE`	Visualizzazione della schermata iniziale per circa 2 secondi.
`TEST_LED`	Tre cicli di accensione e spegnimento degli indicatori luminosi.
`LOGO`	Visualizzazione della schermata identificativa per circa 2 secondi.
`COMPLETATO`	Visualizzazione del messaggio di attesa e ingresso nello stato operativo.

La funzione `gestioneAvvio()` controlla le transizioni attraverso il confronto tra il tempo corrente, restituito da `millis()`, e i timer associati alle varie fasi.

Non vengono impiegate istruzioni `delay()` per determinare la durata delle schermate e del test LED.

Questa soluzione permette al programma di gestire le temporizzazioni senza introdurre attese bloccanti all'interno della funzione.

Durante la fase di avvio, tuttavia, la gestione ordinaria della ricezione UART non viene ancora eseguita.

`PRESENTAZIONE` → `TEST_LED` → `LOGO` → `COMPLETATO` → `OPERATIVO`.

4.7 GESTIONE DELLA RICEZIONE SERIALE

La funzione `riceviDati()` rappresenta il componente principale della comunicazione tra ESP32 e Chain2Gate Engine.

Il suo compito consiste nell'acquisire progressivamente i byte ricevuti attraverso UART2, riconoscere i delimitatori delle trame e verificarne l'integrità.

La ricezione avviene mediante un buffer di memoria e una macchina a stati dedicata.

4.7.1 BUFFER DI RICEZIONE

Il firmware utilizza un array di 384 byte per memorizzare temporaneamente i messaggi ricevuti.

```
uint8_t bufferRx[384];
size_t indiceRx = 0;
uint8_t checksumCalcolato = 0;
```

Il buffer conserva i byte della trama, compresi i delimitatori e il checksum finale.

La variabile `indiceRx` identifica la posizione corrente di scrittura.

La variabile `checksumCalcolato` contiene invece il valore progressivo utilizzato per verificare l'integrità del messaggio.

La dimensione di 384 byte corrisponde alla capacità minima di ricezione raccomandata dalla documentazione del protocollo CCP.

4.7.2 MACCHINA A STATI DELLA RICEZIONE

La logica di ricezione è organizzata attraverso quattro stati.

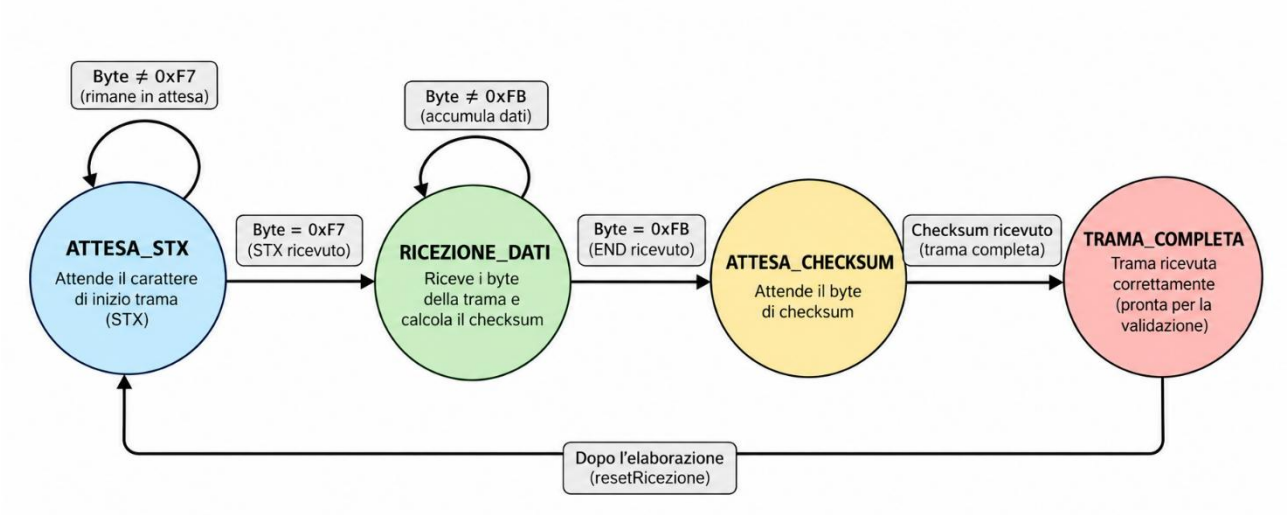
```
enum class StatoRicezione
{
    ATTESA_STX,
    RICEZIONE_DATI,
    ATTESA_CHECKSUM,
    TRAMA_COMPLETA
};
```

Stato	Funzione
`ATTESA_STX`	Attende il carattere di inizio trama `0xF7`.
`RICEZIONE_DATI`	Memorizza i byte e aggiorna il checksum progressivo.
`ATTESA_CHECKSUM`	Acquisisce e verifica il checksum finale.
`TRAMA_COMPLETA`	Segnala la disponibilità di una notifica completa.

Il passaggio tra gli stati avviene in base ai caratteri ricevuti e al risultato della verifica del checksum.

La funzione legge i byte disponibili attraverso `Serial2.read()` e aggiorna progressivamente il contenuto del buffer.

Al riconoscimento di una notifica valida, la ricezione si interrompe temporaneamente per consentire al ciclo principale di elaborare il messaggio.



4.8 VERIFICA DELL'INTEGRITÀ DEI MESSAGGI

Il firmware verifica l'integrità delle trame mediante un checksum a 8 bit.

Il valore viene calcolato come somma modulo 256 dei byte dei dati ricevuti, escludendo i caratteri di delimitazione `STX` ed `ETX`.

Durante la ricezione, ogni byte appartenente ai dati viene sommato al valore progressivo.

```
checksumCalcolato =
    static_cast<uint8_t>(checksumCalcolato + byteRicevuto);
```

L'utilizzo del tipo `uint8_t` permette di conservare gli otto bit meno significativi del risultato, realizzando il calcolo modulo 256.

Dopo la ricezione del delimitatore finale, il programma acquisisce il byte di checksum e lo confronta con quello calcolato.

Se i valori corrispondono, il controllo d'integrità è superato.

Il firmware verifica inoltre che il messaggio provenga dall'indirizzo del Chain2Gate Engine, sia destinato all'Host e riporti il codice della notifica CCP supportata.

Per le notifiche riconosciute, il programma invia una risposta positiva o negativa mediante UART2.

La struttura completa delle trame e i meccanismi ACK/NACK saranno approfonditi nel Capitolo 5.

4.9 GESTIONE DEGLI ERRORI DI RICEZIONE

La comunicazione seriale prevede controlli dedicati alle principali condizioni anomale.

Il firmware gestisce:

- Errori di checksum.
- Timeout di ricezione.
- Superamento della capacità del buffer.
- Ricezione di trame non riconosciute come notifiche CCP supportate.

4.9.1 TIMEOUT DI RICEZIONE

Il firmware utilizza un timeout locale di 300 millisecondi.

Quando una trama è in corso di acquisizione e non arrivano nuovi byte entro l'intervallo previsto, la ricezione viene interrotta e ripristinata.

Il controllo viene effettuato anche quando la porta seriale non contiene dati disponibili.

La temporizzazione utilizza la differenza tra il valore corrente di `millis()` e l'istante di ricezione dell'ultimo byte.

```
millis() - timerUltimoByte >= timeoutRicezione
```

Questa modalità di calcolo consente di gestire correttamente anche il normale rollover del contatore temporale unsigned.

4.9.2 CONTROLLO OVERFLOW

Prima di continuare a memorizzare i dati, il firmware controlla che l'indice di ricezione non abbia raggiunto la capacità del buffer.

Quando viene rilevato un overflow, il programma incrementa il relativo contatore, segnala l'errore e ripristina la ricezione.

4.9.3 CONTATORI DIAGNOSTICI

Per monitorare il comportamento della comunicazione sono previste specifiche variabili.

Variabile	Significato
`trameValide`	Notifiche CCP ricevute con checksum corretto.
`trameIgnorate`	Trame con checksum corretto non riconosciute come notifiche supportate.
`erroriChecksum`	Trame con checksum non corrispondente.
`erroriTimeout`	Ricezioni interrotte per superamento del timeout.
`erroriOverflow`	Eventi di superamento del buffer.
`eventiTotali`	Somma dei contatori diagnostici, calcolata durante l'aggiornamento del monitor seriale.

I contatori vengono visualizzati attraverso la funzione `aggiornaSerialeMonitor()`.

Questi dati permettono di analizzare l'affidabilità della comunicazione durante le prove di laboratorio.

4.10 DECODIFICA DELLE COMPACT FRAME — PARSEFRAME()

Dopo la ricezione di una notifica CCP valida, il firmware richiama la funzione `parseFrame()`.

Questa funzione identifica il tipo di Compact Frame contenuta nel messaggio ed estrae il valore della potenza elettrica istantanea.

La versione attuale riconosce due tipologie:

- **CF21:** notifica associata alla variazione della potenza.
- **CF3:** Compact Frame contenente misurazioni elettriche quartorarie.

Il tipo viene individuato mediante due caratteri ASCII collocati in posizioni definite all'interno del buffer.

4.10.1 IDENTIFICAZIONE DELLA COMPACT FRAME

Il parser legge i caratteri presenti agli indici 10 e 11 del buffer.

```
const char cf1 = static_cast<char>(bufferRx[10]);
const char cf2 = static_cast<char>(bufferRx[11]);
```

Il valore rappresenta il codice della Compact Frame in formato esadecimale testuale.

Compact Frame	Codice ricevuto	Lunghezza trama UART attesa
CF21	"15"	46 byte
CF3	"03"	88 byte

Prima dell'estrazione dei dati viene verificata la lunghezza totale della trama.

4.10.2 ESTRAZIONE DELLA POTENZA

La potenza è rappresentata mediante quattro caratteri esadecimali.

Il parser utilizza posizioni differenti in base alla Compact Frame ricevuta.

Compact Frame	Indici nel buffer
CF21	36–39
CF3	42–45

I caratteri vengono concatenati in una stringa e convertiti in un valore numerico mediante `strtoul()` con base 16.

```
potenzaIstantanea =
    strtoul(potenzaHex.c_str(), nullptr, 16);
```

Il risultato viene memorizzato nella variabile `potenzaIstantanea`, di tipo `uint16_t`.

Questa variabile è condivisa dalle successive funzioni di visualizzazione e trasmissione.

Nella versione corrente, il parser mantiene il valore precedente della potenza se il messaggio non viene riconosciuto oppure presenta una lunghezza inattesa.

Per i messaggi supportati, non è ancora presente un controllo separato sulla validità di ciascun carattere esadecimale prima della conversione.

4.11 TRASMISSIONE DEI DATI VERSO PYTHON

Una volta completata la decodifica, il firmware utilizza la funzione `inviaDatiPython()` per trasmettere la misurazione al computer.

Il trasferimento avviene attraverso la porta seriale USB, utilizzando messaggi in formato JSON (JAVASCRIPT OBJECT NOTATION).

La funzione invia i dati soltanto quando `tipoCf` corrisponde a CF3 oppure CF21.

4.11.1 STRUTTURA DEL MESSAGGIO

Ogni messaggio contiene due campi principali:

Campo	Tipo	Significato
`cf`	Intero	Tipo di Compact Frame riconosciuta.
`potenza_w`	Intero	Potenza istantanea espressa in watt.

Esempio di messaggio trasmesso:

```
{
  "cf": 21,
  "potenza_w": 2448
}
```

Il firmware genera il JSON mediante una sequenza di istruzioni `Serial.print()` e termina il messaggio con `Serial.println()`.

```
Serial.print("{\"cf\":");
Serial.print(tipoCf);

Serial.print(",\"potenza_w\":");
Serial.print(potenzaIstantanea);

Serial.println("}");
```

L'utilizzo di un formato strutturato permette all'applicazione Python di identificare ed estrarre i campi senza interpretare direttamente le trame del protocollo CCP.

La porta USB trasmette anche informazioni diagnostiche testuali. Il software Python distingue i messaggi JSON dalle righe di debug attraverso un controllo sul contenuto ricevuto.

4.12 AGGIORNAMENTO DEL DISPLAY E SEGNALAZIONI LED

Le funzioni `aggiornaDisplay()` e `aggiornaLed()` gestiscono le informazioni rese disponibili attraverso le periferiche locali.

La prima aggiorna il contenuto grafico del display OLED, mostrando il valore della potenza istantanea, il tipo CF e l'indicazione di ricezione.

La seconda controlla gli indicatori luminosi utilizzando timer indipendenti.

Il LED giallo segnala la ricezione di una notifica valida mediante un breve spegnimento, mentre il LED rosso segnala le condizioni di errore gestite.

La funzione `aggiornaLed()` viene richiamata a ogni iterazione dello stato operativo, consentendo di aggiornare le segnalazioni anche quando non sono disponibili nuove trame complete.

4.13 RIPRISTINO DELLA RICEZIONE — RESETRICEZIONE()

Terminata l'elaborazione di una trama, il firmware richiama `resetRicezione()`.

La funzione ripristina le variabili necessarie per iniziare una nuova acquisizione.

```
void resetRicezione()
{
    indiceRx = 0;
```

```
    checksumCalcolato = 0;
    timerUltimoByte = 0;
    nuoviDatiSeriali = false;
    statoRicezione = StatoRicezione::ATTESA_STX;
}
```

Il ripristino non cancella fisicamente il contenuto dell'array `bufferRx`, ma azzera l'indice utilizzato per accedere ai dati della ricezione corrente.

I nuovi byte ricevuti sovrascriveranno progressivamente quelli precedenti.

La separazione di questa operazione in una funzione dedicata permette di richiamare la stessa procedura in seguito a una ricezione completata oppure a un errore che richiede il riavvio logico dell'acquisizione.

4.14 CARATTERISTICHE E LIMITI DELL'IMPLEMENTAZIONE

Il firmware è stato progettato per realizzare un sistema embedded modulare, mantenendo un'organizzazione del codice adatta alle attività di sviluppo e verifica.

Le principali caratteristiche sono:

- Suddivisione delle funzionalità in procedure dedicate.
- Macchine a stati esplicite per la gestione del sistema e della ricezione.
- Utilizzo di timer basati su `millis()` in alternativa a `delay()`.
- Separazione delle porte seriali dedicate al Gate e al computer.
- Verifica delle trame mediante checksum.
- Gestione delle principali anomalie di ricezione.
- Interfaccia locale tramite OLED e indicatori LED.
- Trasmissione delle misurazioni mediante JSON.

L'implementazione presenta anche alcuni limiti da considerare.

La decodifica è circoscritta alle Compact Frame CF3 e CF21 e si concentra sull'estrazione della potenza istantanea.

Il parser non effettua una verifica completa di tutti i campi del payload CCP e utilizza lunghezze fisse per riconoscere le due tipologie supportate.

Lo stato generale `ERRORE` è predisposto ma non dispone ancora di una logica autonoma di gestione.

Inoltre, una notifica CCP formalmente valida ma non riconosciuta dal parser può comunque provocare l'aggiornamento del display con il precedente valore della potenza.

Questi aspetti identificano il perimetro dell'attuale implementazione e costituiscono possibili punti di miglioramento per versioni successive.

4.15 CONCLUSIONI

Il firmware ESP32 costituisce il primo livello di elaborazione del sistema ChainSense2.

Attraverso la gestione della comunicazione UART, il riconoscimento delle trame CCP e la decodifica delle Compact Frame supportate, trasforma i messaggi provenienti dal Chain2Gate Engine in informazioni direttamente utilizzabili.

La suddivisione del programma in funzioni e macchine a stati permette di mantenere separate le responsabilità e facilita l'analisi del comportamento durante le attività di sviluppo e debugging.

L'utilizzo della serializzazione JSON realizza infine il collegamento tra la componente embedded e l'applicazione Python, consentendo alle misurazioni di proseguire verso il database SQLite e la dashboard web.

5. PROTOCOLLO CCP E DECODIFICA DELLE COMPACT FRAME

5.1 INTRODUZIONE AL PROTOCOLLO CCP

La comunicazione tra il microcontrollore ESP32 e il modulo Chain2Gate Engine avviene mediante il protocollo CCP (CHAIN2GATE COMMUNICATION PROTOCOL), attraverso un'interfaccia seriale UART.

Il protocollo consente a un dispositivo Host di comunicare con Chain2Gate Engine, ricevendo le informazioni provenienti dai contatori elettronici Open Meter.

Il modulo Chain2Gate svolge la funzione di interfaccia tra il sistema di comunicazione Chain 2 del contatore e il dispositivo Host. In questo modo, l'ESP32 non deve implementare direttamente le procedure di comunicazione power line e di gestione della cifratura utilizzate dal contatore.

Il protocollo CCP prevede messaggi strutturati, caratterizzati da indirizzi di sorgente e destinazione, un codice identificativo, un payload di lunghezza variabile e un meccanismo di verifica dell'integrità.

Nel progetto ChainSense2 viene utilizzata principalmente la ricezione delle notifiche `CCP_CF_NOTIFICATION`, attraverso le quali Chain2Gate trasmette le Compact Frame contenenti informazioni energetiche.

5.2 ARCHITETTURA DELLA COMUNICAZIONE

La comunicazione CCP è di tipo peer-to-peer: entrambi i dispositivi possono iniziare uno scambio di messaggi.

Nel sistema ChainSense2 sono coinvolti due attori:

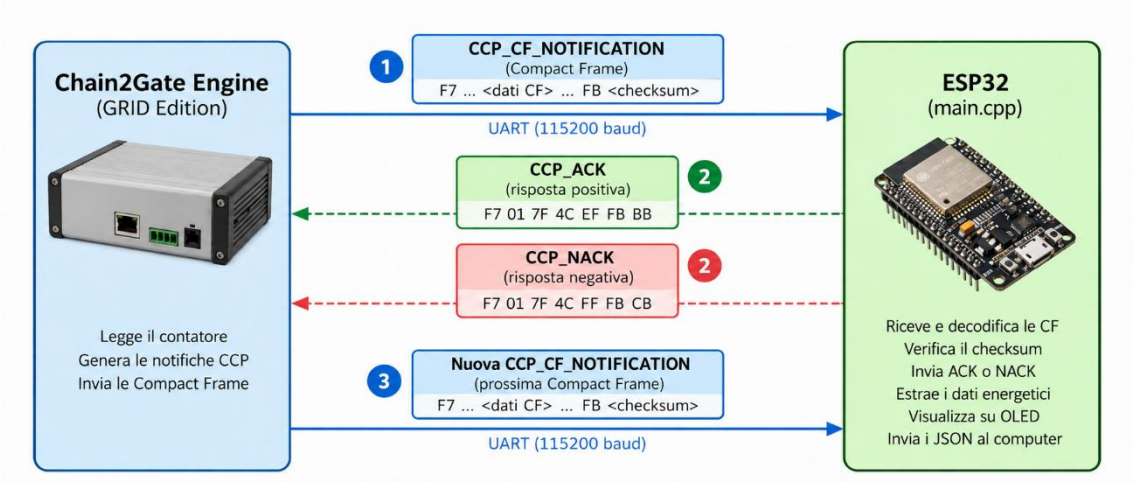
Dispositivo	Indirizzo CCP	Funzione
Chain2Gate Engine	`0x7F`	Trasmette le notifiche provenienti dal contatore
ESP32 (Host)	`0x01`	Riceve, verifica ed elabora i messaggi

La comunicazione avviene attraverso UART2, configurata a 115200 baud, 8 bit di dati, nessuna parità e 1 bit di stop.

Ogni messaggio contiene l'indirizzo del mittente e del destinatario, permettendo di identificare la direzione dello scambio.

Nel normale funzionamento di ChainSense2, Chain2Gate invia una notifica all'ESP32 quando riceve una Compact Frame dal contatore.

L'ESP32 verifica il messaggio e, quando previsto dalla logica implementata, invia una risposta di conferma o di errore.



5.3 STRUTTURA GENERALE DI UNA TRAMA CCP

Ogni messaggio CCP viene trasmesso all'interno di una struttura seriale delimitata da caratteri specifici.

La struttura generale è:

`STX | SOURCE | DESTINATION | MSG_CODE | PAYLOAD | ETX | CHECKSUM`

I campi svolgono le seguenti funzioni:

Campo	Lunghezza	Descrizione
STX	1 byte	Carattere di inizio trama
SOURCE	1 byte	Indirizzo del mittente
DESTINATION	1 byte	Indirizzo del destinatario
MSG_CODE	1 byte	Codice identificativo del messaggio
PAYLOAD	Variabile	Dati associati al messaggio
ETX	1 byte	Carattere di fine dati
CHECKSUM	1 byte	Valore per la verifica dell'integrità

I delimitatori utilizzati dal protocollo sono:

- `STX = 0xF7`
- `ETX = 0xFB`

Il checksum viene trasmesso immediatamente dopo il delimitatore ETX.

Il ricevitore può quindi individuare l'inizio del messaggio, raccogliere i dati, riconoscere la conclusione del payload e verificarne l'integrità.

5.3.1 CODICE IDENTIFICATIVO DEL MESSAGGIO

Il campo `MSG_CODE` specifica la tipologia del messaggio CCP.

Il protocollo prevede diverse operazioni, tra cui richieste di identificazione, configurazione e ricezione delle informazioni energetiche.

ChainSense2 concentra la propria implementazione sul codice:

`0x4C — CCP_CF_NOTIFICATION`

Questo codice identifica una notifica contenente una Compact Frame ricevuta dal contatore elettronico.

Gli altri codici del protocollo non rientrano nel perimetro funzionale del firmware attuale.

5.4 NOTIFICA CCP_CF_NOTIFICATION

La notifica `CCP_CF_NOTIFICATION` viene generata da Chain2Gate quando il modulo riceve una nuova Compact Frame dall'Open Meter.

Il messaggio contiene un'intestazione CCP seguita da una serie di campi che descrivono il contenuto energetico trasmesso.

La struttura del payload può essere rappresentata come:

`TransCnt | Mflag | CFcode | Len | CF Payload`

Campo	Dimensione trasmessa	Funzione
TransCnt	4 caratteri	Contatore progressivo della notifica
Mflag	2 caratteri	Identificativo dell'origine dei dati
CFcode	2 caratteri	Tipo di Compact Frame
Len	2 caratteri	Lunghezza della Compact Frame in byte
CF Payload	Variabile	Informazioni contenute nella Compact Frame

I campi sono rappresentati mediante codifica NIBBLEASCII.

5.4.1 CAMPO TRANSCNT

Il campo `TransCnt` rappresenta un contatore progressivo delle notifiche trasmesse dal modulo Chain2Gate.

È composto da quattro caratteri esadecimali testuali, che rappresentano un valore intero a 16 bit.

Questo campo permette di osservare la progressione delle notifiche ricevute.

Nella versione corrente di ChainSense2 il valore viene acquisito all'interno del buffer, ma non viene utilizzato dal parser per identificare eventuali duplicazioni.

5.4.2 CAMPO MFLAG

Il campo `Mflag` identifica l'origine della Compact Frame.

Il protocollo distingue il contatore di prelievo da eventuali contatori di produzione.

Nel sistema sviluppato, il firmware non utilizza questo campo per filtrare i messaggi in base alla sorgente energetica.

5.4.3 CAMPO CFCODE

Il campo `CFcode` identifica la tipologia della Compact Frame.

È formato da due caratteri esadecimali testuali.

Per le due tipologie supportate:

Codice testuale	Valore esadecimale	Compact Frame
`"15"`	`0x15`	CF21
`"03"`	`0x03`	CF3

È importante distinguere la rappresentazione testuale dal valore numerico: `0x15` corrisponde infatti al numero decimale 21.

5.4.4 CAMPO LEN

Il campo `Len` indica la lunghezza del payload Compact Frame espressa in byte.

Poiché il contenuto viene rappresentato mediante caratteri esadecimali ASCII, ogni byte logico della Compact Frame è trasmesso attraverso due caratteri.

Per esempio, una Compact Frame di 15 byte viene rappresentata attraverso 30 caratteri ASCII.

Nel firmware 1.0 il campo `Len` non viene interpretato e confrontato autonomamente con la dimensione del messaggio: il parser utilizza invece la lunghezza complessiva della trama UART per riconoscere CF21 e CF3.

5.5 CODIFICA NIBBLEASCII

Il protocollo CCP utilizza la rappresentazione NIBBLEASCII per diversi campi del payload.

In questa codifica, ogni cifra esadecimale è trasmessa mediante il corrispondente carattere ASCII.

Per rappresentare un byte sono quindi necessari due caratteri.

Esempio:

```
Valore numerico:        0x15
Rappresentazione:       "15"
Byte trasmessi:         0x31 0x35
```

I caratteri `"1"` e `"5"` hanno infatti codici ASCII rispettivamente `0x31` e `0x35`.

Questa distinzione è fondamentale durante l'analisi dei messaggi.

Una sequenza osservata sul monitor seriale in formato esadecimale potrebbe contenere:

```
31 35
```

Il significato del dato non è rappresentato dai due valori numerici `0x31` e `0x35`, ma dal codice esadecimale testuale `"15"`, che identifica la CF21.

La stessa convenzione viene utilizzata per rappresentare i valori delle misurazioni elettriche.

5.6 ORGANIZZAZIONE DEL BUFFER DI RICEZIONE

Il firmware ChainSense2 memorizza i messaggi in un array denominato `bufferRx`.

L'array contiene la trama UART completa, compresi delimitatori, intestazione, payload e checksum.

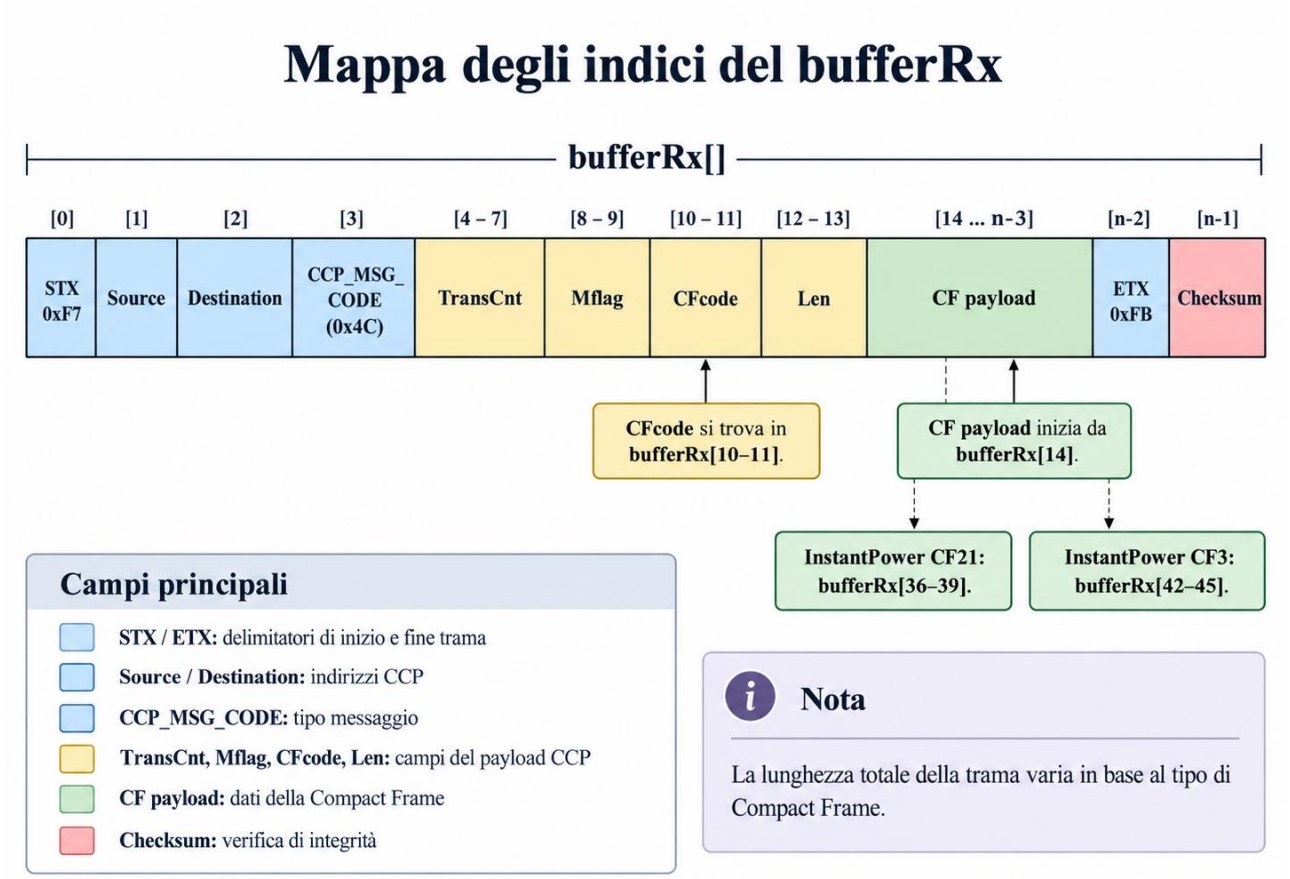
Nel caso di una notifica CCP, i principali campi occupano le seguenti posizioni:

Indice nel buffer	Campo
0	STX
1	SOURCE
2	DESTINATION
3	MSG_CODE
4–7	TransCnt
8–9	Mflag
10–11	CFcode
12–13	Len
Da 14	Payload della Compact Frame

Le posizioni sono calcolate a partire da zero, secondo la normale indicizzazione degli array C++.

Il payload Compact Frame inizia pertanto all'indice 14.

Questo riferimento è utilizzato per determinare la posizione assoluta dei campi energetici all'interno del messaggio.



5.7 VERIFICA DEL CHECKSUM

Il protocollo CCP utilizza un checksum calcolato mediante somma modulo 256 dei byte appartenenti al contenuto del messaggio.

Nel calcolo vengono inclusi indirizzi, codice del messaggio e payload, mentre sono esclusi i delimitatori STX ed ETX e il byte finale di checksum.

La relazione matematica può essere espressa come:

```
Checksum = (somma dei byte dei dati) mod 256
```

Il risultato viene rappresentato mediante un singolo byte.

5.7.1 IMPLEMENTAZIONE NEL FIRMWARE

Durante la ricezione, il firmware aggiorna progressivamente la variabile `checksumCalcolato`.

```
checksumCalcolato =
    static_cast<uint8_t>(checksumCalcolato + byteRicevuto);
```

Quando viene riconosciuto il delimitatore `0xFB`, il programma passa allo stato `ATTESA_CHECKSUM`.

Il byte successivo viene confrontato con il valore calcolato.

```
if (byteRicevuto == checksumCalcolato)
{
    // Checksum corretto
}
else
{
```

```
    // Errore checksum
}
```

Il confronto permette di rilevare numerose alterazioni accidentali del messaggio durante la trasmissione.

Il checksum utilizzato non costituisce tuttavia un meccanismo crittografico e non garantisce l'individuazione di ogni possibile errore.

5.7.2 ESEMPIO DI CALCOLO

Si consideri, a titolo dimostrativo, un messaggio CCP contenente soltanto gli indirizzi e un codice di richiesta:

```
F7 01 7F 4A FB CA
```

I byte da sommare sono:

```
01 + 7F + 4A = CA
```

Il checksum risultante è quindi `0xCA`.

Il byte viene collocato dopo il delimitatore `0xFB`, completando la trama.

5.8 GESTIONE DELLE RISPOSTE ACK E NACK

Il protocollo CCP prevede un sistema di acknowledgment, mediante il quale il destinatario risponde al mittente per confermare o segnalare l'esito di un messaggio.

Le risposte utilizzano due codici:

Codice	Significato
`0xEF`	ACK — Conferma positiva
`0xFF`	NACK — Conferma negativa

La risposta non consiste nel solo byte ACK o NACK, ma viene trasmessa all'interno di una trama CCP completa.

Nel firmware ChainSense2 sono definite due sequenze costanti:

```
constexpr uint8_t tramaAck[] = {
    0xF7, 0x01, 0x7F, 0x4C, 0xEF, 0xFB, 0xBB
};

constexpr uint8_t tramaNack[] = {
    0xF7, 0x01, 0x7F, 0x4C, 0xFF, 0xFB, 0xCB
};
```

Le sequenze includono gli indirizzi dell'Host e del Gate, il codice `0x4C`, il valore dell'acknowledgment e il relativo checksum.

5.8.1 INVIO DELLA RISPOSTA POSITIVA

Quando viene ricevuta una notifica CCP riconoscibile con checksum corretto, il firmware invia la risposta ACK attraverso UART2.

```
Serial2.write(tramaAck, sizeof(tramaAck));
```

La risposta viene trasmessa prima dell'esecuzione del parser delle Compact Frame.

5.8.2 INVIO DELLA RISPOSTA NEGATIVA

In caso di checksum errato su una notifica CCP riconoscibile, il firmware trasmette la sequenza NACK.

```
Serial2.write(tramaNack, sizeof(tramaNack));
```

Contestualmente viene incrementato il contatore degli errori e attivata la segnalazione luminosa prevista dal firmware.

Per le altre tipologie di messaggio CCP non riconosciute, la versione attuale non implementa un gestore generale delle risposte.

5.8.3 TIMEOUT E RITRASMISSIONI

Secondo le specifiche CCP del modulo Chain2Gate Engine, il dispositivo attende la risposta del destinatario entro una finestra di 600 millisecondi.

Se non riceve la conferma prevista, può effettuare una ritrasmissione del messaggio, secondo la configurazione del protocollo.

Questa temporizzazione deve essere distinta dal timeout locale di 300 millisecondi utilizzato dal firmware ESP32 per individuare una ricezione seriale incompleta.

I due timeout operano su livelli differenti della comunicazione.

5.9 COMPACT FRAME CF21

La CF21 è una Compact Frame associata a un evento di variazione della potenza elettrica prelevata.

Secondo la documentazione del dispositivo, la notifica viene generata quando la variazione della potenza istantanea supera una soglia configurata.

Questa caratteristica permette di ottenere aggiornamenti in corrispondenza di variazioni significative del carico elettrico.

5.9.1 STRUTTURA DEL PAYLOAD CF21

Il payload della CF21 contiene 30 caratteri NIBBLEASCII, equivalenti a 15 byte logici.

La struttura comprende le seguenti informazioni:

Campo	Caratteri	Offset nel payload CF
MessagePosixTimestamp	8	0–7
EventType	2	8–9
EventPosixTimestamp	8	10–17
EventsCounter	4	18–21
InstantPower	4	22–25
PowerThreshold	4	26–29

Il campo di interesse per ChainSense2 è `InstantPower`, che rappresenta la potenza attiva istantanea espressa in watt.

5.9.2 ESTRAZIONE DELLA POTENZA CF21

Poiché il payload CF inizia all'indice 14 del buffer UART e il campo InstantPower inizia all'offset 22, la posizione assoluta della potenza risulta:

```
14 + 22 = 36
```

Il valore viene quindi letto dagli indici 36–39.

Il firmware esegue la seguente elaborazione:

```
String potenzaHex = "";

for (int i = 36; i < 40; i++)
{
    potenzaHex += static_cast<char>(bufferRx[i]);
}

potenzaIstantanea =
    strtoul(potenzaHex.c_str(), nullptr, 16);
```

La stringa contenente quattro caratteri esadecimali viene convertita in un numero intero.

Per esempio:

```
Valore esadecimale: 0x0990
Valore decimale:    2448
Potenza:            2448 W
```

5.9.3 LUNGHEZZA DELLA TRAMA CF21

La CF21 contiene 30 caratteri di payload energetico.

A questi si aggiungono i campi dell'intestazione CCP, i delimitatori e il checksum.

La lunghezza complessiva della notifica UART utilizzata dal firmware è di 46 byte.

Il parser accetta il riconoscimento della CF21 soltanto quando il codice ricevuto corrisponde a `"15"` e la lunghezza totale è pari a 46 byte.

5.10 COMPACT FRAME CF3

La CF3 è una Compact Frame contenente misurazioni elettriche quartorarie.

La struttura documentata per questa tipologia si riferisce al profilo trifase P4 del contatore Open Meter M1.

Rispetto alla CF21, il payload contiene un insieme più ampio di informazioni, comprendente misure energetiche e valori di potenza.

ChainSense2 utilizza esclusivamente il campo relativo alla potenza istantanea.

5.10.1 STRUTTURA DEL PAYLOAD CF3

La CF3 contiene 72 caratteri NIBBLEASCII, equivalenti a 36 byte logici.

Tra i campi presenti si trovano:

Campo	Caratteri	Offset nel payload CF
MessagePosixTimestamp	8	0–7

MeasurePosixTimestamp	8	8–15
TariffCode	2	16–17
Array elements	2	18–19
CurrQuartActEnergy	4	20–23
PrevQuartActEnergy	4	24–27
InstantPower	4	28–31
QuartAveragePower	4	32–35
Altri campi	36	36–71

Il campo InstantPower occupa quattro caratteri esadecimali, analogamente a quanto previsto per la CF21.

5.10.2 ESTRAZIONE DELLA POTENZA CF3

Nel payload CF3 il campo InstantPower inizia all'offset 28.

Considerando l'inizio del payload nell'indice 14 del buffer:

```
14 + 28 = 42
```

Il firmware legge pertanto i caratteri presenti agli indici 42–45.

```
String potenzaHex = "";

for (int i = 42; i < 46; i++)
{
    potenzaHex += static_cast<char>(bufferRx[i]);
}

potenzaIstantanea =
    strtoul(potenzaHex.c_str(), nullptr, 16);
```

La conversione produce il valore numerico della potenza istantanea, che viene memorizzato nella stessa variabile utilizzata per CF21.

La lunghezza complessiva della notifica UART CF3 attesa dal firmware è di 88 byte.

5.11 CONFRONTO TRA CF21 E CF3

Le due Compact Frame forniscono informazioni con caratteristiche differenti, ma condividono il campo relativo alla potenza istantanea.

Caratteristica	CF21	CF3
Codice esadecimale	0x15	0x03

Tipo di informazione	Evento variazione potenza	Misure quartorarie
Payload CF	30 caratteri	72 caratteri
Lunghezza trama UART	46 byte	88 byte
Offset InstantPower nella CF	22–25	28–31
Indici InstantPower nel buffer	36–39	42–45
Unità della potenza	Watt	Watt

La scelta di supportare entrambe le tipologie permette a ChainSense2 di elaborare la potenza istantanea proveniente da due diverse strutture informative.

Le notifiche non devono tuttavia essere considerate campioni acquisiti a intervalli regolari.

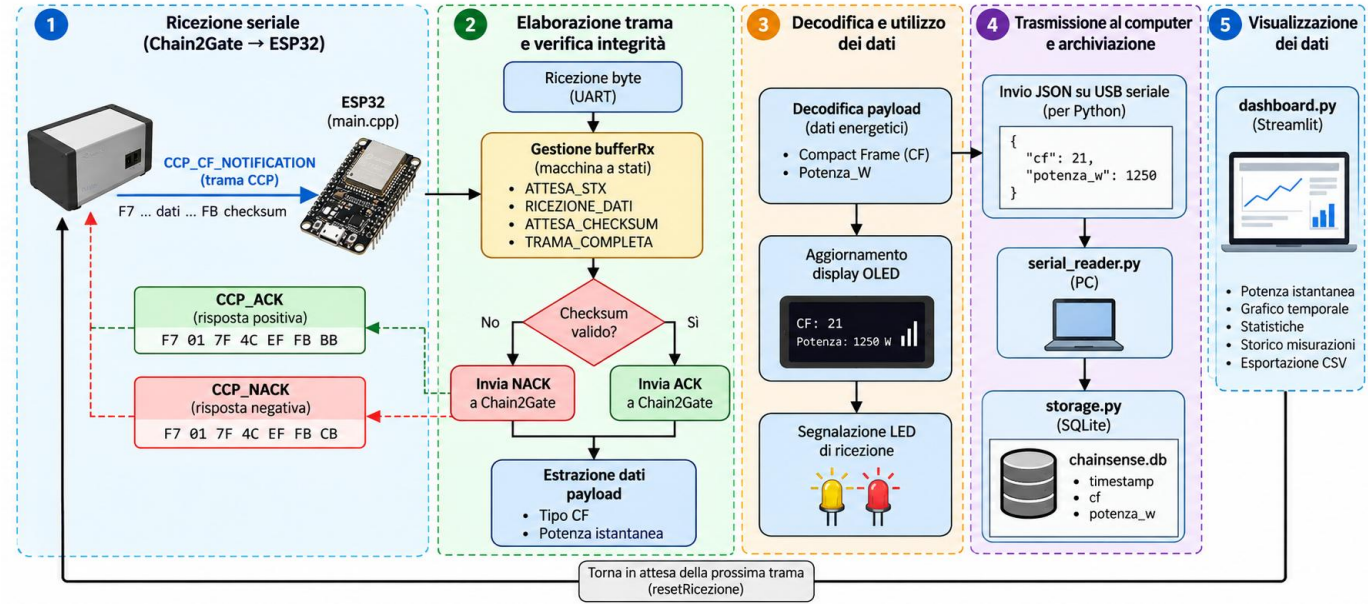
La CF21 è associata a un evento di variazione della potenza, mentre la CF3 contiene informazioni relative alle misurazioni quartorarie.

La frequenza delle notifiche dipende quindi dal comportamento e dalla configurazione del sistema di misura.

5.12 FLUSSO COMPLETO DI ELABORAZIONE DI UNA NOTIFICA

Il processo di gestione di una notifica CCP può essere suddiviso nelle seguenti operazioni:

1. Chain2Gate invia una notifica attraverso UART.
2. ESP32 riconosce il delimitatore iniziale `0xF7`.
3. Il firmware memorizza progressivamente i byte ricevuti.
4. Il delimitatore `0xFB` segnala la conclusione dei dati.
5. Il checksum viene confrontato con quello calcolato.
6. Il firmware verifica gli indirizzi e il codice CCP.
7. Una notifica riconoscibile con checksum corretto riceve un ACK.
8. Il parser identifica il tipo CF e verifica la lunghezza prevista.
9. Se la CF è supportata, viene estratta la potenza istantanea.
10. Il valore viene trasmesso in formato JSON.
11. Il display OLED viene aggiornato.
12. Il sistema ripristina la ricezione per il messaggio successivo.



5.13 LIMITI DELL'IMPLEMENTAZIONE DEL PROTOCOLLO

Il firmware ChainSense2 implementa un sottoinsieme delle funzionalità disponibili nel protocollo CCP.

L'obiettivo è acquisire e interpretare le informazioni necessarie al monitoraggio della potenza istantanea, senza realizzare un Host CCP completo.

I principali limiti dell'implementazione sono:

- Supporto limitato alle notifiche di tipo `CCP_CF_NOTIFICATION`.
- Decodifica circoscritta alle Compact Frame CF21 e CF3.
- Mancata interpretazione completa dei campi informativi presenti nei payload.
- Controllo delle dimensioni basato sulla lunghezza totale attesa della trama.
- Assenza di un controllo autonomo di coerenza del campo `Len`.
- Assenza di una validazione completa dei caratteri NIBBLEASCII prima della conversione.
- Mancata gestione delle notifiche duplicate mediante il contatore `TransCnt`.
- Risposte ACK/NACK implementate esclusivamente per le notifiche riconoscibili dal firmware.

È inoltre importante distinguere la validità del messaggio CCP dal riconoscimento della Compact Frame.

Una notifica può superare il controllo checksum ed essere confermata mediante ACK, senza necessariamente contenere una CF supportata dal parser.

Il firmware, infatti, effettua il riconoscimento della tipologia energetica soltanto dopo la verifica e l'eventuale conferma della trama CCP.

5.14 CONCLUSIONI

Il protocollo CCP costituisce il collegamento tra il modulo Chain2Gate Engine e il firmware embedded di ChainSense2.

La comprensione della struttura dei messaggi, della codifica NIBBLEASCII, dei delimitatori e dei meccanismi ACK/NACK è fondamentale per ricevere e interpretare correttamente le informazioni energetiche.

Il sistema sviluppato utilizza queste caratteristiche per identificare le notifiche CCP, verificarne l'integrità e decodificare le Compact Frame CF21 e CF3.

L'estrazione della potenza istantanea rappresenta il risultato principale di questa elaborazione.

Il valore ottenuto viene successivamente trasmesso in formato JSON all'applicazione Python, che si occupa dell'acquisizione e dell'archiviazione delle misurazioni.

Questa separazione consente di mantenere la complessità del protocollo CCP all'interno del firmware ESP32, fornendo ai componenti software successivi dati già interpretati e strutturati.

6. COMUNICAZIONE ESP32–PYTHON E ACQUISIZIONE DATI

6.1 INTRODUZIONE

Il sistema ChainSense2 utilizza un'applicazione Python per acquisire le misurazioni elaborate dal firmware ESP32 e trasferirle al livello di archiviazione.

Questa componente costituisce il collegamento tra il sistema embedded e l'applicazione software eseguita sul computer.

Dopo aver ricevuto e decodificato le Compact Frame provenienti dal Chain2Gate Engine, l'ESP32 trasmette le informazioni di interesse attraverso la porta seriale USB, utilizzando messaggi strutturati in formato JSON.

Il programma Python, contenuto nel file `serial_reader.py`, si occupa di leggere i messaggi ricevuti, interpretarli, estrarre i valori numerici e associarli a un riferimento temporale.

Le misurazioni vengono successivamente trasferite al modulo `storage.py`, responsabile della persistenza nel database SQLite.

Questa organizzazione consente di mantenere separate le responsabilità di elaborazione embedded, acquisizione software e archiviazione.

6.2 ARCHITETTURA DELLA COMUNICAZIONE

La comunicazione tra ESP32 e computer avviene mediante collegamento USB, attraverso l'interfaccia seriale esposta dal microcontrollore.

Il firmware utilizza due porte seriali con funzioni differenti:

Interfaccia	Utilizzo
`Serial2`	Comunicazione UART tra ESP32 e Chain2Gate Engine
`Serial`	Trasmissione dei dati JSON e dei messaggi diagnostici al computer

Questa separazione permette di utilizzare il protocollo CCP sulla porta UART2, mantenendo la comunicazione con l'applicazione Python sulla porta seriale USB.

I dati trasmessi al computer non contengono l'intera Compact Frame, ma soltanto le informazioni già interpretate dal firmware.

Il programma Python non deve quindi conoscere la struttura del protocollo CCP né gli offset dei campi contenuti nei messaggi energetici.



6.3 FORMATO DEI DATI TRASMESSI

Per lo scambio delle informazioni tra ESP32 e Python è stato adottato il formato JSON (JAVASCRIPT OBJECT NOTATION).

JSON è un formato testuale per la rappresentazione di dati strutturati attraverso coppie chiave-valore.

La sua struttura permette di identificare i campi attraverso nomi espliciti, facilitando la comunicazione tra applicazioni sviluppate con linguaggi differenti.

6.3.1 STRUTTURA DEL MESSAGGIO JSON

Il firmware ChainSense2 trasmette un messaggio contenente due informazioni:

Campo	Tipo	Descrizione
`cf`	Intero	Identificativo della Compact Frame riconosciuta
`potenza_w`	Intero	Potenza istantanea espressa in watt

Esempio di messaggio:

```
{
  "cf": 21,
  "potenza_w": 2448
}
```

Il campo `cf` permette di distinguere le misurazioni provenienti da CF3 e CF21.

Il campo `potenza_w` contiene invece il valore numerico della potenza istantanea, già convertito dalla rappresentazione esadecimale utilizzata nel protocollo CCP.

6.3.2 GENERAZIONE DEL MESSAGGIO SULL'ESP32

La funzione `inviaDatiPython()` costruisce il messaggio JSON e lo trasmette sulla porta `Serial`.

Il firmware utilizza istruzioni `Serial.print()` per scrivere progressivamente i campi e `Serial.println()` per terminare il messaggio.

```
Serial.print("{\"cf\":");
Serial.print(tipoCf);

Serial.print(",\"potenza_w\":");
Serial.print(potenzaIstantanea);

Serial.println("}");
```

La trasmissione viene effettuata esclusivamente per le Compact Frame riconosciute dal parser, CF3 e CF21.

Il carattere di fine riga generato da `Serial.println()` permette all'applicazione Python di distinguere i singoli messaggi durante la lettura seriale.

6.4 ORGANIZZAZIONE DEL MODULO SERIAL_READER.PY

Il file `serial_reader.py` costituisce il componente principale dell'acquisizione software.

Il suo funzionamento è organizzato intorno alla funzione `main()`, che coordina l'inizializzazione del database, l'apertura della porta seriale e la ricezione continua dei messaggi.

Le responsabilità principali dello script sono:

1. Inizializzare il sistema di archiviazione.
2. Aprire la porta seriale associata all'ESP32.
3. Leggere i messaggi ricevuti.
4. Distinguere i dati JSON dai messaggi diagnostici.
5. Decodificare il contenuto JSON.
6. Estrarre e convertire i valori numerici.
7. Associare un timestamp alla misurazione.
8. Richiamare il modulo di archiviazione.
9. Gestire le principali eccezioni.

La struttura mantiene la logica di acquisizione separata dalle operazioni SQL, che vengono eseguite nel modulo `storage.py`.

6.5 LIBRERIE E MODULI UTILIZZATI

Lo script utilizza librerie standard di Python e il pacchetto esterno PySerial.

Modulo	Funzione
`json`	Decodifica dei messaggi JSON

`serial`	Gestione della comunicazione seriale mediante PySerial
`datetime`	Generazione del riferimento temporale
`storage`	Inizializzazione e salvataggio delle misurazioni

Le importazioni presenti nel codice sono:

```
import json
import serial

from datetime import datetime
from storage import salva_misurazione, inizializza_database
```

Il modulo `json` appartiene alla libreria standard di Python, mentre PySerial deve essere installato separatamente nell'ambiente in cui viene eseguito il lettore.

Il modulo `storage` è un componente locale del progetto ChainSense2.

6.6 CONFIGURAZIONE DELLA PORTA SERIALE

La configurazione della comunicazione seriale è definita mediante due costanti.

```
PORTA = "COM3"
VELOCITA = 115200
```

Parametro	Valore configurato
Porta seriale	COM3
Velocità	115200 baud
Timeout di lettura	1 secondo
Modalità di acquisizione	Lettura per righe

Il nome `COM3` identifica la porta seriale utilizzata nell'ambiente Windows di sviluppo.

Su altri computer il dispositivo potrebbe essere associato a una porta differente, rendendo necessaria la modifica del parametro `PORTA`.

6.6.1 APERTURA DELLA CONNESSIONE

La porta seriale viene aperta attraverso la classe `Serial` del pacchetto PySerial.

```
with serial.Serial(PORTA, VELOCITA, timeout=1) as esp32:
    print("Connessione ESP32 avviata...")
```

Il costrutto `with` permette di gestire il ciclo di vita della connessione.

Quando il blocco termina, la porta seriale viene chiusa attraverso il meccanismo di gestione del contesto fornito dalla libreria.

Il parametro `timeout=1` limita a un secondo l'attesa delle operazioni di lettura.

Questa temporizzazione non corrisponde alla frequenza di acquisizione delle misurazioni: i nuovi dati dipendono dai messaggi effettivamente trasmessi dall'ESP32.

6.7 CICLO DI ACQUISIZIONE DEI MESSAGGI

Dopo l'apertura della connessione, lo script entra in un ciclo continuo mediante l'istruzione `while True`.

A ogni iterazione, il programma tenta di acquisire una riga dalla porta seriale.

```
messaggio = esp32.readline().decode(
    "utf-8", errors="replace"
).strip()
```

L'istruzione comprende tre operazioni principali.

Lettura seriale — readline()

Il metodo legge i byte disponibili fino al raggiungimento del terminatore di riga oppure alla scadenza del timeout configurato.

Decodifica — decode()

I byte ricevuti vengono convertiti in una stringa Python utilizzando la codifica UTF-8.

Il parametro `errors="replace"` permette di sostituire eventuali sequenze non valide, evitando un'interruzione immediata della decodifica testuale.

Normalizzazione — strip()

Il metodo elimina gli spazi e i caratteri di fine riga presenti all'inizio e alla fine della stringa.

Il risultato è un messaggio testuale pronto per le successive verifiche.

6.8 SEPARAZIONE DEI DATI DAI MESSAGGI DIAGNOSTICI

La porta seriale USB viene utilizzata dal firmware sia per trasmettere le misurazioni JSON sia per visualizzare informazioni diagnostiche.

Il software Python deve quindi distinguere i messaggi destinati all'archiviazione dalle altre righe di testo.

Nel codice viene utilizzato il seguente controllo:

```
if not messaggio.startswith("{"):
    continue
```

Il metodo `startswith()` verifica se la stringa ricevuta inizia con il carattere `{`, utilizzato come delimitatore iniziale degli oggetti JSON.

Quando la condizione non è soddisfatta, l'istruzione `continue` interrompe l'iterazione corrente e riprende il ciclo di acquisizione.

Questo meccanismo permette di ignorare le righe diagnostiche generate dal firmware.

Il controllo costituisce un filtro preliminare: non garantisce da solo che il contenuto sia un JSON valido. La verifica sintattica viene effettuata successivamente attraverso `json.loads()`.

6.9 DECODIFICA E INTERPRETAZIONE DEL JSON

Una volta individuato un messaggio potenzialmente valido, il programma utilizza il modulo `json` per interpretarne il contenuto.

```
dati = json.loads(messaggio)
```

Il metodo `json.loads()` converte la stringa JSON in un oggetto Python.

Nel caso dei messaggi trasmessi da ChainSense2, il risultato atteso è un dizionario contenente le chiavi `cf` e `potenza_w`.

6.9.1 ESTRAZIONE DEI VALORI

I dati vengono recuperati mediante l'accesso alle chiavi del dizionario.

```
tipo_cf = int(dati["cf"])
potenza = int(dati["potenza_w"])
```

La funzione `int()` converte i valori estratti in numeri interi.

Il controllo permette di ottenere dati numerici utilizzabili dalle successive funzioni di archiviazione.

La versione corrente non applica ulteriori vincoli applicativi, come un intervallo massimo di potenza o una verifica esplicita che `cf` appartenga esclusivamente ai valori 3 e 21.

La selezione delle Compact Frame supportate viene effettuata principalmente nel firmware ESP32.

6.10 ASSOCIAZIONE DEL TIMESTAMP

Ogni misurazione viene associata a un riferimento temporale generato dal computer mediante la funzione `datetime.now()`.

```
timestamp = datetime.now()
```

Il timestamp rappresenta la data e l'ora del sistema host nel momento in cui Python elabora il messaggio ricevuto.

È importante distinguere questo valore dal timestamp eventualmente presente all'interno della Compact Frame originale.

Nell'implementazione corrente, il firmware trasmette solamente il tipo CF e la potenza istantanea.

Il riferimento temporale utilizzato per l'archiviazione viene quindi generato dall'applicazione Python, senza recuperare il timestamp interno del contatore.

Questa scelta permette di mantenere semplice il formato JSON, ma introduce una dipendenza dall'orologio del computer e dal momento effettivo di acquisizione.

6.11 TRASFERIMENTO DEI DATI AL MODULO STORAGE

Dopo la decodifica del messaggio e la generazione del timestamp, `serial_reader.py` richiama la funzione `salva_misurazione()`.

```
salva_misurazione(timestamp, tipo_cf, potenza)
```

La funzione appartiene al modulo `storage.py` e riceve tre parametri.

Parametro	Tipo previsto	Significato
`timestamp`	datetime	Data e ora di acquisizione
`tipo_cf`	int	Identificativo della Compact Frame
`potenza`	int	Potenza istantanea in watt

Il lettore seriale non esegue direttamente operazioni SQL.

La gestione del database rimane affidata al modulo di persistenza.

Questa separazione permette di modificare il sistema di archiviazione senza riscrivere necessariamente la procedura di ricezione seriale.

6.12 INIZIALIZZAZIONE DEL DATABASE

Prima di avviare la lettura seriale, la funzione `main()` richiama:

```
inizializza_database()
```

Questa operazione prepara il livello di persistenza, creando la cartella e la tabella necessarie nel caso in cui non siano già presenti.

L'inizializzazione viene eseguita una sola volta all'avvio dello script, prima dell'apertura della porta seriale.

Le modalità di creazione della tabella, gestione delle connessioni e salvataggio dei record vengono approfondite nel Capitolo 7.

6.13 GESTIONE DELLE ECCEZIONI

La comunicazione con dispositivi esterni e l'elaborazione dei dati possono generare errori.

Per evitare che ogni dato non valido interrompa immediatamente l'acquisizione, lo script utilizza blocchi `try` ed `except`.

6.13.1 ERRORI DI DECODIFICA JSON

La libreria `json` può generare un'eccezione `JSONDecodeError` quando il messaggio ricevuto non rispetta la sintassi prevista.

```
except json.JSONDecodeError as errore:
    print(f"Errore decodifica JSON: {errore}")
```

L'errore viene segnalato sul terminale e il ciclo prosegue con il messaggio successivo.

6.13.2 ERRORI NEI DATI RICEVUTI

Il programma gestisce inoltre tre tipologie di eccezione legate all'interpretazione dei campi.

Eccezione	Possibile causa
`KeyError`	Campo richiesto assente nel dizionario
`TypeError`	Tipo di dato incompatibile con la conversione
`ValueError`	Valore non convertibile in intero

Il codice utilizza un unico blocco per intercettare queste condizioni.

```
except (KeyError, TypeError, ValueError) as errore:
    print(f"Errore nei dati ricevuti: {errore}")
```

6.13.3 ERRORI DELLA COMUNICAZIONE SERIALE

L'eccezione `serial.SerialException` viene utilizzata per gestire problemi relativi all'apertura o all'utilizzo della porta seriale.

Tra le possibili cause rientrano una porta non disponibile, un dispositivo scollegato o una risorsa seriale già occupata.

Nella versione attuale, un'eccezione seriale non determina un tentativo automatico di riconnessione: il programma segnala l'errore e termina il ciclo di acquisizione.

6.13.4 ERRORI DI ACCESSO AI FILE

L'eccezione `OSError` permette di intercettare errori di input/output, tra cui eventuali problemi di accesso al file di archiviazione.

Il programma visualizza il messaggio di errore sul terminale.

6.13.5 INTERRUZIONE MANUALE

L'eccezione `KeyboardInterrupt` viene intercettata per gestire l'interruzione richiesta dall'utente, normalmente mediante la combinazione di tasti `Ctrl+C`.

Questa gestione consente di terminare lo script mostrando un messaggio informativo.

6.14 INFORMAZIONI DIAGNOSTICHE SUL TERMINALE

Dopo aver trasferito una misurazione al modulo di archiviazione, lo script visualizza le informazioni relative ai dati acquisiti.

L'output comprende:

- Tipo Compact Frame.
- Valore della potenza istantanea.
- Timestamp associato.
- Conferma del completamento della chiamata di salvataggio.

Esempio di visualizzazione:

```
CF ricevuta: 21
Potenza: 2448 W
Timestamp: 2026-10-29 12:20:00
Misurazione salvata.
-----------------------------
```

I valori dell'esempio sono dimostrativi.

La diagnostica permette di verificare che il programma abbia ricevuto e interpretato il messaggio prima di consultare direttamente il database SQLite.

6.15 AVVIO DELL'APPLICAZIONE PYTHON

Il file `serial_reader.py` prevede il seguente blocco conclusivo:

```
if __name__ == "__main__":
    main()
```

La variabile speciale `__name__` permette di distinguere l'esecuzione diretta dello script dal suo utilizzo come modulo importato da un altro programma.

Quando il file viene eseguito direttamente, la condizione risulta vera e viene richiamata la funzione `main()`.

6.15.1 COMANDO DI ESECUZIONE

Dopo aver installato Python e la libreria PySerial, il lettore può essere avviato dal terminale mediante:

```
python serial_reader.py
```

Prima dell'esecuzione è necessario verificare che:

- L'ESP32 sia collegato al computer.

- Il firmware sia in esecuzione.
- La porta seriale configurata corrisponda a quella assegnata al dispositivo.
- La porta non sia occupata da un'altra applicazione.
- Il modulo `storage.py` sia disponibile nella directory del progetto.

Una volta avviato, il programma rimane in ascolto e acquisisce i messaggi trasmessi dall'ESP32.

6.16 LIMITI DELL'IMPLEMENTAZIONE

Il modulo di acquisizione è stato realizzato per una configurazione prototipale con un singolo dispositivo ESP32 collegato al computer.

L'implementazione presenta alcune caratteristiche da considerare:

- La porta seriale è configurata direttamente nel codice.
- Non è previsto il riconoscimento automatico del dispositivo.
- Non è implementata una procedura automatica di riconnessione.
- Il programma utilizza un singolo processo di acquisizione.
- Il timestamp viene generato dal computer.
- La validazione dei dati è limitata alla decodifica JSON e alla conversione dei campi richiesti.
- Non sono implementati controlli applicativi per rilevare misurazioni duplicate.
- La porta seriale USB trasporta contemporaneamente messaggi diagnostici e dati JSON.
- La persistenza richiede che il database sia accessibile dal processo Python.

Queste caratteristiche rispondono alle esigenze della versione attuale e individuano possibili aree di sviluppo per applicazioni più complesse.

6.17 CONCLUSIONI

Il modulo `serial_reader.py` costituisce il collegamento tra il firmware embedded e il sistema di archiviazione delle misurazioni.

La comunicazione mediante JSON permette di trasferire dati già interpretati, mantenendo il software Python indipendente dai dettagli del protocollo CCP.

La separazione tra acquisizione e persistenza rende l'architettura più ordinata e facilita l'evoluzione dei singoli componenti.

Attraverso la lettura seriale, la decodifica dei messaggi, la gestione delle eccezioni e l'associazione dei timestamp, il programma trasforma le informazioni trasmesse dal microcontrollore in misurazioni pronte per essere archiviate e successivamente visualizzate dalla dashboard ChainSense2.

7. DATABASE SQLITE E PERSISTENZA DELLE MISURAZIONI

7.1 INTRODUZIONE

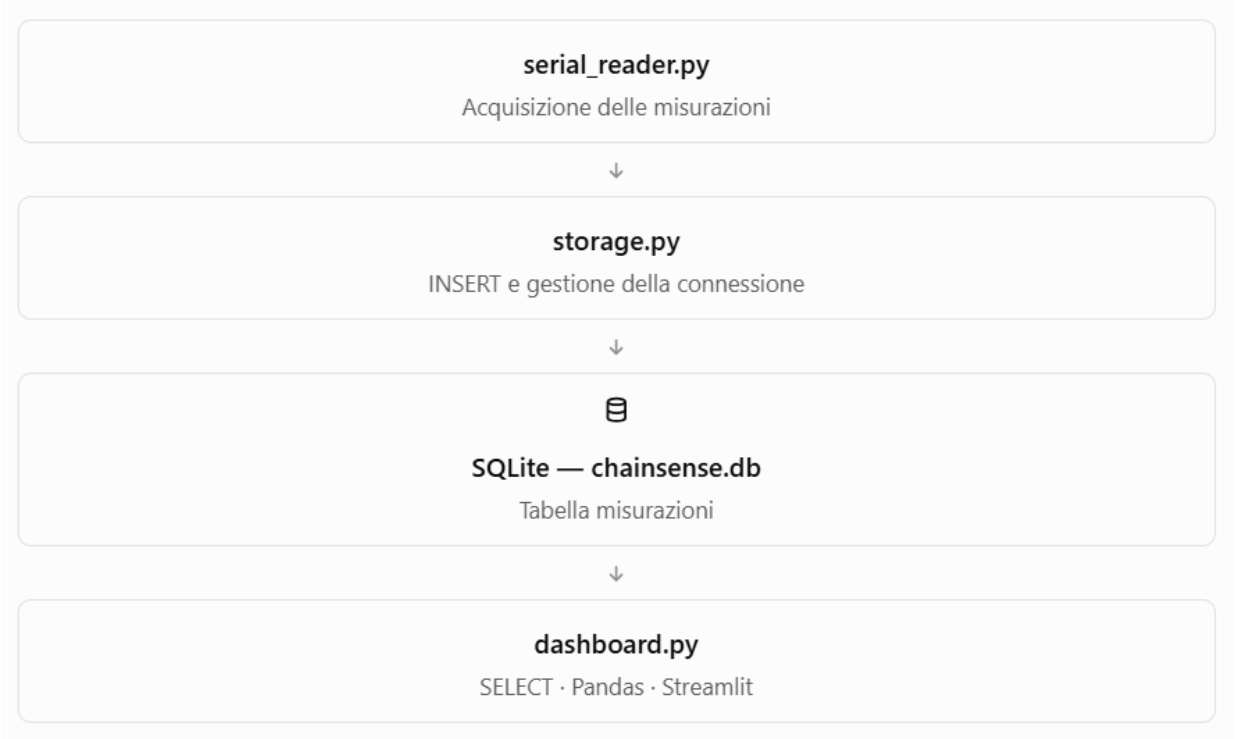
ChainSense2 utilizza un database SQLite per archiviare in modo persistente le misurazioni energetiche ricevute dall'ESP32.

Il database costituisce il livello di memorizzazione del sistema e permette di conservare lo storico delle letture indipendentemente dall'esecuzione della dashboard web.

La gestione dell'archiviazione è affidata al modulo Python `storage.py`, che si occupa dell'inizializzazione del database, dell'inserimento delle misurazioni e della lettura dei record.

SQLite è un sistema di gestione di database relazionali leggero, che non richiede l'installazione di un server dedicato e memorizza i dati all'interno di un file.

Nel progetto ChainSense2 il database è denominato `chainsense.db` e si trova nella directory `data` dell'applicazione.



7.2 ARCHITETTURA DEL LIVELLO DI PERSISTENZA

Il sistema mantiene separate le responsabilità di acquisizione, archiviazione e visualizzazione.

Il modulo `serial_reader.py` riceve i dati dall'ESP32 e richiama `storage.py` per registrare ogni misurazione.

La dashboard Streamlit accede successivamente allo stesso database, attraverso una connessione SQLite dedicata, per recuperare le informazioni necessarie alla visualizzazione.

I componenti coinvolti sono:

Componente	Responsabilità
`serial_reader.py`	Ricezione dei JSON e generazione del timestamp
`storage.py`	Creazione del database e inserimento delle misurazioni
`chainsense.db`	Conservazione persistente dei record
`dashboard.py`	Interrogazione ed elaborazione dei dati archiviati

Questa separazione permette di eseguire l'acquisizione e la dashboard come processi indipendenti.

7.3 ORGANIZZAZIONE DEL DATABASE

La versione attuale di ChainSense2 utilizza un'unica tabella relazionale, denominata `misurazioni`.

Ogni record rappresenta una misurazione acquisita dall'applicazione Python.

7.3.1 STRUTTURA DELLA TABELLA

Campo	Tipo SQLite	Descrizione
`id`	INTEGER PRIMARY KEY	Identificativo univoco della misurazione
`timestamp`	TEXT	Data e ora di acquisizione
`cf`	INTEGER	Tipo di Compact Frame, CF3 o CF21
`potenza_w`	INTEGER	Potenza istantanea in watt

La definizione della tabella è contenuta nel modulo `storage.py`.

```
CREATE TABLE IF NOT EXISTS misurazioni (
    id INTEGER PRIMARY KEY,
    timestamp TEXT,
    cf INTEGER,
    potenza_w INTEGER
);
```

Il campo `id` identifica univocamente ogni record e viene assegnato automaticamente da SQLite quando non viene specificato nell'operazione di inserimento.

Il campo `timestamp` conserva il riferimento temporale generato da Python, mentre `cf` e `potenza_w` contengono le informazioni estratte dal messaggio JSON.

La struttura non prevede attualmente tabelle aggiuntive o relazioni tra entità differenti.

7.3.2 ESEMPIO DI RECORD

id	timestamp	cf	potenza_w
1	2026-10-29T10:15:00	21	2448
2	2026-10-29T10:16:30	21	2510
3	2026-10-29T10:30:00	3	2498

TABELLA ESEMPLIFICATIVA: VALORI UTILIZZATI PER ILLUSTRARE LA STRUTTURA DEI DATI.

Ogni misurazione è indipendente dalle altre e può essere recuperata successivamente mediante interrogazioni SQL.

7.4 MODULO STORAGE.PY

Il file `storage.py` contiene le funzioni dedicate alla gestione del database SQLite.

Il modulo utilizza tre componenti della libreria standard di Python:

```
import sqlite3
from datetime import datetime
from pathlib import Path
```

La libreria `sqlite3` permette di comunicare con il database ed eseguire istruzioni SQL.

`datetime` viene utilizzato per la rappresentazione del riferimento temporale, mentre `Path` consente di costruire il percorso del file in modo indipendente dalla directory corrente del terminale.

7.4.1 PERCORSO DEL DATABASE

Il percorso viene definito mediante la costante:

```
PERCORSO_DB = (
    Path(__file__).resolve().parent
    / "data"
    / "chainsense.db"
)
```

Il percorso viene costruito rispetto alla directory in cui si trova `storage.py`.

Questa scelta permette di individuare il database anche quando lo script viene avviato da una directory differente.

7.5 INIZIALIZZAZIONE DEL DATABASE

L'inizializzazione viene gestita dalla funzione `inizializza_database()`.

La funzione viene richiamata da `serial_reader.py` prima dell'apertura della connessione seriale.

Le operazioni comprendono:

1. Verifica e creazione della cartella `data`.
2. Apertura o creazione del file SQLite.
3. Creazione della tabella `misurazioni`, se assente.
4. Conferma delle operazioni mediante `commit()`.

Il codice principale è:

```
def inizializza_database():
    PERCORSO_DB.parent.mkdir(
        parents=True, exist_ok=True
    )

    with sqlite3.connect(PERCORSO_DB) as connessione:
        connessione.execute("""
            CREATE TABLE IF NOT EXISTS misurazioni (
                id INTEGER PRIMARY KEY,
                timestamp TEXT,
                cf INTEGER,
                potenza_w INTEGER
            )
        """)
```

```
        connessione.commit()
```

La clausola `IF NOT EXISTS` impedisce di ricreare una tabella già esistente, preservando i record archiviati.

Il blocco `with` gestisce la conferma o l'annullamento delle transazioni al termine del contesto. Non garantisce, da solo, la chiusura immediata della connessione SQLite.

7.6 INSERIMENTO DELLE MISURAZIONI

Il salvataggio è affidato alla funzione `salva_misurazione()`, che riceve i dati elaborati da `serial_reader.py`.

La funzione accetta tre parametri:

- `timestamp`: data e ora di acquisizione.
- `tipo_cf`: identificativo della Compact Frame.
- `potenza`: valore della potenza istantanea in watt.

L'inserimento viene effettuato attraverso un'istruzione SQL parametrizzata.

```
connessione.execute("""
    INSERT INTO misurazioni (
        timestamp, cf, potenza_w
    )
    VALUES (?, ?, ?)
""", (
    timestamp.isoformat(timespec="seconds"),
    tipo_cf,
    potenza
))

connessione.commit()
```

I segnaposto `?` consentono di separare i valori dall'istruzione SQL, evitando la costruzione della query mediante concatenazione di stringhe.

Il timestamp viene convertito in formato ISO 8601 con precisione al secondo.

Al termine dell'inserimento, `commit()` conferma la modifica nel database.

7.6.1 GESTIONE DELLA CONNESSIONE

La funzione apre una connessione SQLite e utilizza un blocco `try/finally` per garantirne la chiusura.

```
connessione = sqlite3.connect(PERCORSO_DB)

try:
    # Inserimento e conferma della misurazione
    ...
finally:
    connessione.close()
```

Il blocco `finally` viene eseguito anche in presenza di un'eccezione, permettendo di rilasciare la connessione.

Nell'implementazione attuale viene aperta una nuova connessione per ciascuna operazione di salvataggio.

7.7 LETTURA DELLE MISURAZIONI

Il modulo `storage.py` include anche la funzione `leggi_misurazioni()`, utilizzabile per recuperare i record presenti nella tabella.

La funzione esegue la query:

```
SELECT * FROM misurazioni;
```

Il risultato viene recuperato mediante `fetchall()`, che restituisce le righe sotto forma di una lista di tuple Python.

Anche questa funzione utilizza `try/finally` per garantire la chiusura della connessione.

La dashboard non richiama direttamente `leggi_misurazioni()`, ma utilizza una propria funzione di caricamento basata su Pandas.

7.7.1 ACCESSO DALLA DASHBOARD

Nel file `dashboard.py`, la funzione `carica_misurazioni()` interroga SQLite mediante `pd.read_sql_query()`.

```
dati = pd.read_sql_query("""
    SELECT timestamp AS Timestamp,
           cf AS CF,
           potenza_w AS Potenza_W
    FROM misurazioni
    ORDER BY timestamp ASC
""", connessione)
```

La query recupera i campi necessari alla dashboard e li ordina cronologicamente.

L'istruzione `AS` assegna alle colonne del risultato i nomi utilizzati nelle successive operazioni Pandas, senza modificare la struttura originale del database.

Il risultato viene trasformato direttamente in un DataFrame, utilizzato per grafici, statistiche e tabelle.

7.8 PERSISTENZA E CONSERVAZIONE DEI DATI

L'utilizzo di SQLite permette di conservare le misurazioni anche dopo la chiusura del programma Python o della dashboard.

I record rimangono memorizzati nel file `chainsense.db`, fino a quando non vengono modificati o eliminati mediante operazioni esplicite.

Nel sistema ChainSense2 il database si trova sul computer host Windows.

La dashboard eseguita nel container Docker accede al file attraverso un collegamento alla directory `data`, configurato in sola lettura.

Questa architettura permette di mantenere la persistenza indipendente dal ciclo di vita del container.

La presenza del database sul sistema host non costituisce tuttavia una procedura di backup: per proteggere lo storico è necessario prevedere copie separate del file.

7.9 LIMITI E POSSIBILI EVOLUZIONI

La struttura del database è stata progettata per le esigenze dell'attuale prototipo.

L'implementazione presenta alcune limitazioni:

- Utilizzo di un'unica tabella dedicata alle misurazioni.
- Assenza di relazioni tra più entità.
- Mancanza di vincoli applicativi sui valori `cf` e `potenza_w`.
- Memorizzazione del timestamp con precisione al secondo.
- Assenza di meccanismi di identificazione automatica delle misurazioni duplicate.
- Mancanza di una procedura automatizzata di backup e conservazione dello storico.

La struttura potrebbe essere estesa per supportare ulteriori informazioni energetiche, più dispositivi di acquisizione o interrogazioni più articolate.

Eventuali modifiche dovrebbero mantenere la separazione tra acquisizione, persistenza e visualizzazione, già adottata nell'architettura del progetto.

7.10 CONCLUSIONI

Il database SQLite costituisce il livello di persistenza di ChainSense2 e permette di trasformare le misurazioni ricevute in uno storico consultabile.

Il modulo `storage.py` centralizza le operazioni di inizializzazione, inserimento e lettura dei dati, mantenendo indipendente la logica di acquisizione seriale.

L'utilizzo di query parametrizzate, connessioni dedicate e un formato temporale coerente rende la struttura semplice da comprendere e da mantenere.

La separazione tra il database e la dashboard consente infine di conservare i dati sul sistema host e consultarli attraverso l'interfaccia Streamlit, anche quando quest'ultima viene eseguita in ambiente Docker.

8. DASHBOARD WEB STREAMLIT

8.1 INTRODUZIONE

ChainSense2 integra un'interfaccia web sviluppata in Python mediante il framework Streamlit, destinata alla consultazione e all'analisi delle misurazioni energetiche archiviate.

La dashboard costituisce il livello di presentazione del sistema e permette all'utente di visualizzare la potenza istantanea, consultare lo storico delle acquisizioni e analizzare l'andamento dei valori mediante grafici e statistiche.

L'applicazione accede direttamente al database SQLite, senza richiedere una comunicazione seriale con il microcontrollore ESP32.

Il codice sorgente è contenuto nel file `dashboard.py`, che organizza le funzionalità attraverso procedure dedicate alla lettura, elaborazione e rappresentazione dei dati.

8.2 TECNOLOGIE UTILIZZATE

La dashboard utilizza principalmente Streamlit, Pandas e SQLite.

Tecnologia	Funzione
Streamlit	Creazione dell'interfaccia web e gestione dei componenti grafici

Pandas	Organizzazione, filtraggio e analisi delle misurazioni
SQLite	Accesso allo storico dei dati
datetime	Gestione dei riferimenti e degli intervalli temporali
pathlib	Individuazione del percorso del database

Le librerie vengono importate all'inizio del programma.

```
from pathlib import Path
from datetime import datetime, timedelta

import pandas as pd
import streamlit as st
import sqlite3
```

Streamlit permette di realizzare un'interfaccia web direttamente dal codice Python, senza sviluppare separatamente un frontend HTML e JavaScript.

Pandas viene utilizzato per elaborare i risultati delle interrogazioni SQL e trasformarli in strutture tabellari denominate DataFrame.

8.3 ORGANIZZAZIONE DEL CODICE

Il file `dashboard.py` è suddiviso in funzioni, ciascuna responsabile di una specifica operazione.

Funzione	Responsabilità
`carica_misurazioni()`	Recupera i dati dal database SQLite
`mostra_potenza()`	Visualizza l'ultima misura e gli indicatori principali
`filtra_misurazioni()`	Seleziona le misurazioni in base all'intervallo temporale
`mostra_grafico()`	Rappresenta l'andamento della potenza
`mostra_statistiche()`	Calcola e visualizza minimo, media e massimo
`mostra_tabella()`	Mostra le ultime dieci misurazioni selezionate
`esporta_csv()`	Genera il file CSV scaricabile
`mostra_footer()`	Visualizza le informazioni identificative del progetto
`aggiorna_monitoraggio()`	Coordina l'interfaccia e il suo aggiornamento periodico

La separazione delle responsabilità consente di mantenere il codice organizzato e facilita eventuali modifiche alle singole sezioni.

8.4 CONFIGURAZIONE E INTERFACCIA GRAFICA

L'applicazione viene configurata attraverso `st.set_page_config()`.

```
st.set_page_config(
    page_title="ChainSense2",
    page_icon="⚡",
    layout="wide"
)
```

La modalità `wide` consente di sfruttare maggiormente lo spazio orizzontale disponibile nel browser.

L'interfaccia presenta un'intestazione identificativa del progetto, seguita dalle metriche principali, dai grafici, dallo storico e dagli strumenti di esportazione.

8.4.1 PERSONALIZZAZIONE GRAFICA

Il layout utilizza componenti Streamlit e regole CSS personalizzate inserite mediante `st.markdown()`.

La personalizzazione comprende:

- Colori e sfondi dell'interfaccia.
- Aspetto delle metriche numeriche.
- Barra grafica della potenza.
- Spaziatura e disposizione dei componenti.
- Stile di pulsanti, tabelle e selettori.

La struttura grafica è pensata per evidenziare le informazioni principali e mantenere separati i diversi blocchi di monitoraggio.



8.5 CARICAMENTO DELLE MISURAZIONI

La funzione `carica_misurazioni()` accede al database SQLite e recupera i campi necessari all'interfaccia.

Il risultato dell'interrogazione viene caricato direttamente in un DataFrame Pandas mediante `pd.read_sql_query()`.

```
dati = pd.read_sql_query("""
    SELECT timestamp AS Timestamp,
           cf AS CF,
```

```
            potenza_w AS Potenza_W
    FROM misurazioni
    ORDER BY timestamp ASC
""", connessione)
```

L'ordinamento crescente permette di mantenere le misurazioni in sequenza cronologica.

Dopo il caricamento, i timestamp vengono convertiti in valori temporali gestibili da Pandas.

```
dati["Timestamp"] = pd.to_datetime(
    dati["Timestamp"],
    errors="coerce"
)

dati = dati.dropna(subset=["Timestamp"])
```

Eventuali timestamp non interpretabili vengono esclusi dal DataFrame.

Se il file del database non esiste oppure l'interrogazione restituisce un risultato vuoto, la funzione ritorna un DataFrame senza misurazioni.

Questa gestione evita di eseguire le successive elaborazioni su un archivio privo di dati. Non comprende tuttavia una gestione dedicata di tutte le possibili anomalie del database.

8.6 VISUALIZZAZIONE DELLA POTENZA

La funzione `mostra_potenza()` presenta le informazioni principali relative alle misurazioni archiviate.

Le metriche visualizzate comprendono:

- Ultima potenza istantanea disponibile.
- Potenza massima registrata nello storico caricato.
- Tipo di Compact Frame dell'ultima misurazione.
- Data e ora dell'ultima acquisizione.

Il valore più recente viene recuperato dal DataFrame ordinato cronologicamente.

```
ultima_misura = dati.iloc[-1]

potenza = ultima_misura["Potenza_W"]
potenza_massima = dati["Potenza_W"].max()
```

Le informazioni numeriche vengono rappresentate mediante il componente `st.metric()`.

8.6.1 INDICATORE DELLA POTENZA

La dashboard include una barra di progresso che rappresenta il rapporto tra la potenza misurata e un valore di riferimento configurato.

Nella versione corrente il riferimento è fissato a 6000 W.

```
potenza_riferimento = 6000

percentuale = potenza / potenza_riferimento * 100

progresso = min(
    max(percentuale / 100, 0.0),
```

```
    1.0
)
```

La funzione `st.progress()` rappresenta graficamente il rapporto calcolato.

Il valore viene limitato all'intervallo compreso tra 0 e 1, necessario per la visualizzazione della barra.

La potenza di 6000 W costituisce un riferimento scelto per il prototipo e non deve essere interpretata automaticamente come potenza contrattuale dell'impianto.

8.7 STATO DELL'ULTIMA MISURAZIONE

La dashboard calcola il tempo trascorso dall'ultima acquisizione confrontando il timestamp registrato con l'orologio del computer.

```
secondi_trascorsi = (
    datetime.now() - ultima_ricezione
).total_seconds()
```

Il risultato determina il messaggio di stato visualizzato.

Tempo trascorso	Indicazione
Inferiore a 30 secondi	Misurazione recente
Da 30 a meno di 120 secondi	Misurazione non recente
Da 120 secondi in poi	Misurazione obsoleta

Le soglie sono definite nella logica applicativa della dashboard.

L'indicatore rappresenta l'età dell'ultima misurazione disponibile e non costituisce una verifica diretta della connessione fisica con il contatore.

In particolare, una CF21 può non essere trasmessa per un periodo prolungato in assenza delle variazioni di potenza che ne determinano la generazione.

8.8 FILTRAGGIO TEMPORALE

La dashboard permette all'utente di selezionare l'intervallo temporale da analizzare.

Gli intervalli disponibili sono:

Selezione	Durata
Ultimi 5 minuti	5 minuti
Ultimi 15 minuti	15 minuti
Ultimi 30 minuti	30 minuti
Ultimi 60 minuti	60 minuti

Il selettore è realizzato mediante `st.selectbox()`.

La funzione `filtra_misurazioni()` calcola l'inizio dell'intervallo e seleziona dal DataFrame i record con timestamp successivo o uguale.

```
def filtra_misurazioni(dati, intervallo):
    inizio_intervallo = (
        datetime.now() - timedelta(minutes=intervallo)
    )

    return dati[
        dati["Timestamp"] >= inizio_intervallo
    ]
```

Il filtro viene applicato ai grafici, alle statistiche, alla tabella e all'esportazione CSV.

Le metriche principali relative all'ultima potenza e alla potenza massima vengono invece calcolate sul DataFrame completo.

8.9 RAPPRESENTAZIONE GRAFICA DELLE MISURAZIONI

La dashboard utilizza due rappresentazioni grafiche principali.

8.9.1 GRAFICO DELL'ANDAMENTO DELLA POTENZA

La funzione `mostra_grafico()` visualizza l'evoluzione della potenza istantanea nel tempo mediante un grafico a linee.

```
st.line_chart(
    dati,
    x="Timestamp",
    y="Potenza_W",
    x_label="Orario",
    y_label="Potenza (W)"
)
```

L'asse orizzontale rappresenta il timestamp di acquisizione, mentre quello verticale indica la potenza in watt.

Il grafico permette di osservare le variazioni registrate durante l'intervallo selezionato.

Le misurazioni non vengono necessariamente acquisite a intervalli costanti, poiché la loro disponibilità dipende dalla ricezione delle Compact Frame.



8.9.2 STATISTICHE DELLA POTENZA

La funzione `mostra_statistiche()` calcola tre indicatori sui campioni selezionati:

- Potenza minima.
- Potenza media.
- Potenza massima.

Le elaborazioni vengono effettuate mediante i metodi Pandas `.min()`, `.mean()` e `.max()`.

I risultati sono rappresentati attraverso un grafico a barre realizzato con `st.bar_chart()`.

La media calcolata è una media aritmetica dei valori registrati, non una media ponderata rispetto alla durata degli intervalli tra le acquisizioni.

Le statistiche rappresentano quindi l'insieme dei campioni disponibili e non costituiscono un calcolo dell'energia consumata.

8.10 CONSULTAZIONE DELLO STORICO

La funzione `mostra_tabella()` permette di consultare le ultime dieci misurazioni appartenenti all'intervallo temporale selezionato.

I record vengono recuperati mediante `tail(10)` e ordinati dal più recente al meno recente.

```
ultime_10 = dati.tail(10)

ultime_10 = ultime_10.sort_values(
    "Timestamp",
    ascending=False
)

st.dataframe(
    ultime_10,
    hide_index=True,
    width="stretch"
)
```

La tabella mostra timestamp, tipologia della Compact Frame e valore della potenza.

L'utilizzo di `st.dataframe()` permette di presentare le informazioni in un componente tabellare interattivo.

8.11 ESPORTAZIONE DEI DATI CSV

La dashboard include una funzione per esportare le misurazioni in formato CSV.

Il file viene generato direttamente dal DataFrame Pandas e reso disponibile mediante un pulsante Streamlit.

```
csv = (
    dati.tail(10)
    .to_csv(index=False, sep=";")
    .encode("utf-8-sig")
)
```

L'esportazione utilizza:

- Separatore `;` tra i campi.
- Codifica UTF-8 con BOM.
- Esclusione dell'indice Pandas.

La codifica scelta favorisce la compatibilità con applicazioni di foglio elettronico, anche in ambienti con impostazioni locali italiane.

Il file viene generato con il nome:

`ChainSense2_misurazioni.csv`

Nella versione corrente l'esportazione comprende soltanto le ultime dieci misurazioni dell'intervallo selezionato, non l'intero storico del database.



8.12 AGGIORNAMENTO PERIODICO DELLA DASHBOARD

L'interfaccia utilizza un meccanismo di aggiornamento periodico basato sui fragment di Streamlit.

La funzione principale è decorata nel seguente modo:

```
@st.fragment(run_every="3s")
def aggiorna_monitoraggio():
    dati = carica_misurazioni()

    if dati.empty:
        st.warning("Nessuna misurazione disponibile.")
        return

    # Aggiornamento dei componenti
```

Il decoratore `@st.fragment(run_every="3s")` permette di rieseguire periodicamente la funzione di monitoraggio durante una sessione attiva.

A ogni aggiornamento vengono recuperate le misurazioni dal database e aggiornati i componenti dell'interfaccia.

L'intervallo di tre secondi riguarda il refresh della dashboard e non la frequenza di trasmissione dei dati da parte del contatore o dell'ESP32.

La dashboard mostra quindi i dati più recenti disponibili nell'archivio, senza richiedere un accesso diretto alla comunicazione UART.

8.13 COORDINAMENTO DELL'INTERFACCIA

La funzione `aggiorna_monitoraggio()` costituisce il punto centrale dell'applicazione.

Coordina le seguenti operazioni:

1. Caricamento delle misurazioni.
2. Verifica della disponibilità dei dati.
3. Visualizzazione della potenza e dei relativi indicatori.
4. Selezione dell'intervallo temporale.
5. Applicazione del filtro.
6. Visualizzazione dei grafici e delle statistiche.
7. Consultazione delle ultime misurazioni.
8. Esportazione in formato CSV.
9. Visualizzazione delle informazioni finali del progetto.

Se il database non contiene misurazioni, viene presentato un messaggio informativo.

Se invece sono presenti dati nello storico ma non nell'intervallo selezionato, la dashboard mantiene visibili le metriche generali e segnala l'assenza di nuove misurazioni per il periodo richiesto.

La disposizione dei componenti grafici viene realizzata attraverso `st.columns()`, consentendo di affiancare il grafico temporale e le statistiche.

8.14 LIMITI DELL'IMPLEMENTAZIONE

La dashboard ChainSense2 è stata sviluppata come interfaccia prototipale di monitoraggio.

La versione attuale presenta alcune limitazioni:

- Il caricamento dei dati recupera l'intera tabella delle misurazioni.
- I filtri disponibili sono limitati a intervalli temporali predefiniti.
- Non è previsto un filtro separato per CF3 e CF21.
- L'esportazione CSV è limitata agli ultimi dieci record filtrati.
- La potenza di riferimento è definita direttamente nel codice.
- Non è prevista una gestione completa delle eccezioni relative alle interrogazioni SQLite.
- Il sistema non implementa il calcolo dell'energia consumata in kWh.
- Le informazioni sullo stato della comunicazione si basano sull'età dell'ultima misurazione, senza monitorare direttamente la connessione con l'ESP32.

Questi limiti costituiscono possibili aree di miglioramento per successive versioni del software.

8.15 CONCLUSIONI

La dashboard Streamlit rappresenta il livello di presentazione del sistema ChainSense2.

Attraverso l'integrazione di Python, Pandas e SQLite, l'applicazione permette di trasformare le misurazioni archiviate in informazioni consultabili tramite browser.

La struttura modulare del codice distingue le attività di caricamento, filtraggio, elaborazione e visualizzazione.

L'utilizzo di grafici, indicatori numerici, tabelle e strumenti di esportazione permette di consultare sia il valore più recente della potenza istantanea sia lo storico delle acquisizioni.

L'aggiornamento periodico consente infine di visualizzare le nuove misurazioni rese disponibili dal sistema di acquisizione, mantenendo la dashboard indipendente dalla comunicazione seriale dell'ESP32.

9. DOCKER E CONTAINERIZZAZIONE

9.1 INTRODUZIONE

ChainSense2 utilizza Docker per eseguire la dashboard web Streamlit all'interno di un ambiente Linux containerizzato, separato dal sistema operativo host.

La containerizzazione permette di distribuire l'applicazione insieme al runtime Python e alle librerie necessarie, riducendo la dipendenza dalla configurazione software del computer sul quale viene eseguita.

L'implementazione adottata prevede la containerizzazione del solo componente di visualizzazione, mentre il programma responsabile dell'acquisizione seriale e della scrittura nel database SQLite rimane in esecuzione direttamente su Windows.

Questa scelta permette di evitare la gestione dell'accesso diretto alla porta USB seriale da parte del container Linux e di mantenere distinte le responsabilità dei processi.

Il deployment viene definito attraverso quattro file:

File	Responsabilità
`Dockerfile`	Definisce la costruzione dell'immagine applicativa
`requirements.txt`	Elenca le dipendenze Python della dashboard
`.dockerignore`	Esclude file e directory non necessari dal contesto di build
`compose.yaml`	Configura il servizio, il networking e il collegamento ai dati persistenti

9.2 ARCHITETTURA DI ESECUZIONE

L'ambiente di sviluppo e di esecuzione del prototipo è basato su Windows, Docker Desktop e backend WSL 2.

Docker Desktop utilizza un ambiente Linux per eseguire i container, mentre i comandi Docker vengono impartiti dall'host attraverso PowerShell.

L'architettura di ChainSense2 prevede due processi applicativi distinti.

Processo di acquisizione — Windows

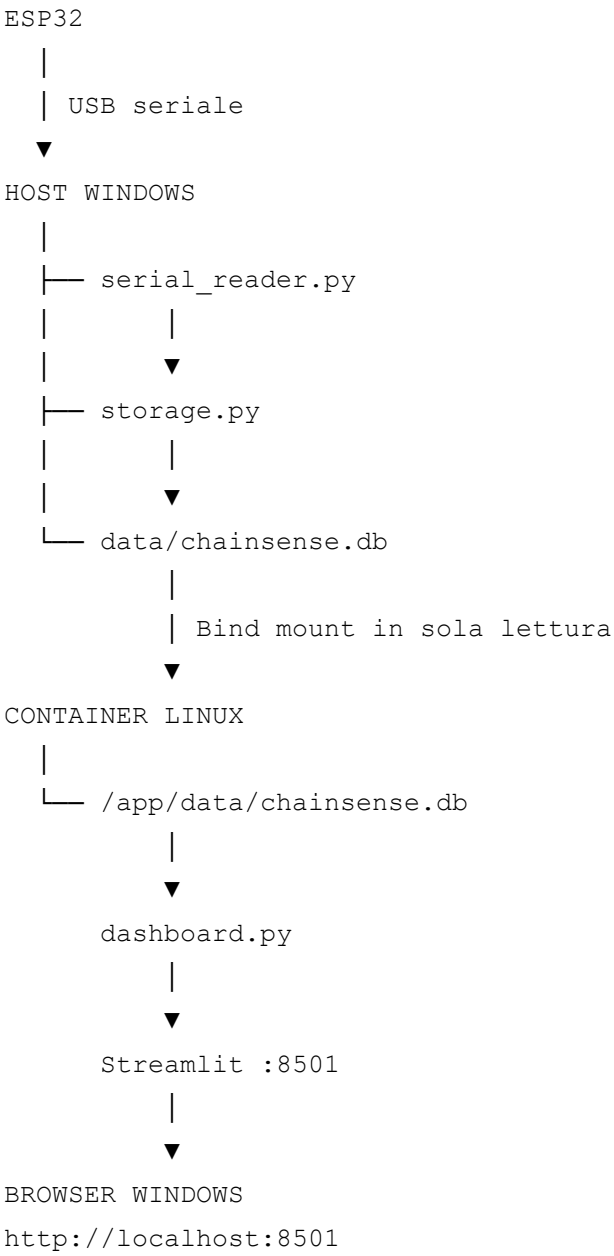
Il programma `serial_reader.py` apre la porta seriale dell'ESP32, riceve i messaggi JSON e richiama `storage.py` per inserire le misurazioni nel database.

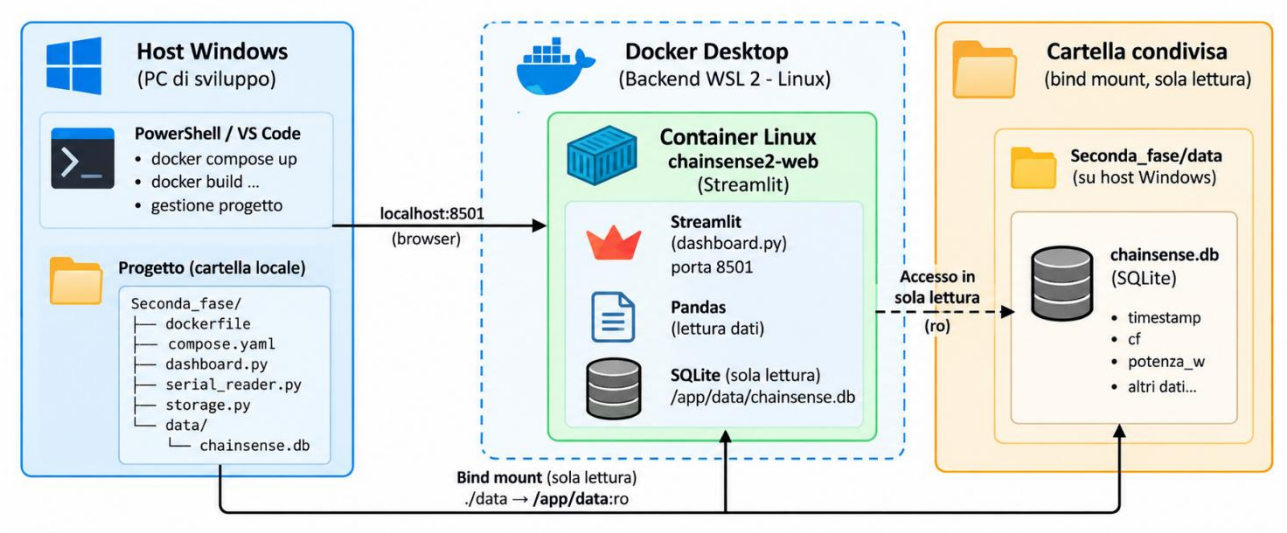
Processo di visualizzazione — Container Linux

Il programma `dashboard.py` viene eseguito all'interno di un container Docker, interroga il database SQLite e rende disponibile l'interfaccia web attraverso il server Streamlit.

Il database viene condiviso tra i due ambienti mediante un bind mount.

9.2.1 FLUSSO DEI COMPONENTI





9.3 IMMAGINI E CONTAINER DOCKER

Docker distingue il concetto di immagine da quello di container.

Un'immagine Docker è un insieme di livelli e metadati utilizzato come riferimento per creare un ambiente applicativo.

Un container è un'istanza eseguibile di un'immagine, dotata di un proprio ambiente di processo e filesystem.

Nel caso di ChainSense2:

Elemento	Identificativo
Immagine applicativa	`chainsense2-dashboard:1.1`
Container	`chainsense2-web`
Directory di lavoro	`/app`
Porta applicativa	`8501`
Applicazione eseguita	`dashboard.py`

L'immagine contiene il runtime Python, le dipendenze software e il codice della dashboard.

Il database operativo non viene incluso nell'immagine, ma viene reso disponibile al container attraverso un collegamento alla directory dell'host.

Questa separazione permette di ricostruire o sostituire il container mantenendo lo storico delle misurazioni sul computer.

9.4 ANALISI DEL DOCKERFILE

Il file `Dockerfile` contiene le istruzioni utilizzate da Docker per costruire l'immagine applicativa.

La configurazione adottata è:

```
FROM python:3.12-slim
```

```
WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY dashboard.py .

EXPOSE 8501

CMD ["streamlit", "run", "dashboard.py", "--server.address=0.0.0.0"]
```

9.4.1 IMMAGINE DI BASE — FROM

```
FROM python:3.12-slim
```

L'istruzione `FROM` seleziona l'immagine di base utilizzata durante la build.

La configurazione utilizza Python 3.12 nella variante `slim`, che contiene un ambiente Linux ridotto rispetto a immagini Python più complete.

L'utilizzo di una base alleggerita permette di limitare le componenti non necessarie all'esecuzione della dashboard.

Il tag `3.12-slim` identifica una famiglia di immagini e non blocca automaticamente una specifica revisione immutabile.

9.4.2 DIRECTORY DI LAVORO — WORKDIR

```
WORKDIR /app
```

L'istruzione imposta `/app` come directory di lavoro per le operazioni successive.

I percorsi relativi utilizzati dalle istruzioni `COPY` e dal comando di avvio fanno quindi riferimento a questa directory.

9.4.3 INSTALLAZIONE DELLE DIPENDENZE

```
COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt
```

Il file delle dipendenze viene copiato nell'immagine prima del codice applicativo.

Successivamente, l'istruzione `RUN` installa i pacchetti Python richiesti.

Questa organizzazione consente di sfruttare il sistema di cache dei livelli Docker: se il file `requirements.txt` non cambia, il livello di installazione delle dipendenze può essere riutilizzato durante le build successive.

L'opzione `--no-cache-dir` disabilita la cache di download di pip, senza disabilitare la cache dei livelli Docker.

9.4.4 COPIA DELL'APPLICAZIONE

```
COPY dashboard.py .
```

Questa istruzione copia il sorgente della dashboard nella directory `/app` dell'immagine.

Non vengono copiati i moduli `serial_reader.py` e `storage.py`, perché il container non è responsabile dell'acquisizione seriale né dell'inserimento delle misurazioni.

La dashboard utilizza direttamente SQLite attraverso il modulo standard `sqlite3`.

9.4.5 PORTA E COMANDO DI AVVIO

```
EXPOSE 8501


CMD ["streamlit", "run", "dashboard.py", "--server.address=0.0.0.0"]
```

`EXPOSE 8501` documenta la porta TCP utilizzata dall'applicazione. Non pubblica autonomamente la porta sul computer host.

L'istruzione `CMD` definisce invece il comando predefinito eseguito all'avvio del container.

Il parametro `--server.address=0.0.0.0` configura Streamlit affinché ascolti sulle interfacce di rete del container, rendendo possibile l'accesso attraverso il port mapping Docker.

9.5 GESTIONE DELLE DIPENDENZE — REQUIREMENTS.TXT

Il file `requirements.txt` contiene le librerie Python necessarie all'esecuzione della dashboard nel container.

```
streamlit
pandas
```

Le dipendenze sono limitate ai pacchetti esterni effettivamente utilizzati dal livello di visualizzazione.

I moduli `sqlite3`, `datetime` e `pathlib` appartengono alla libreria standard di Python e non richiedono un'installazione separata tramite pip.

Il pacchetto `pyserial`, necessario al processo `serial_reader.py`, deve invece essere disponibile nell'ambiente Python dell'host Windows.

Non è incluso nell'immagine Docker, perché il lettore seriale non viene eseguito nel container.

Nella configurazione corrente le dipendenze non sono vincolate a versioni specifiche. Di conseguenza, build effettuate in momenti differenti possono installare versioni differenti dei pacchetti.

9.6 ESCLUSIONE DEI FILE — .DOCKERIGNORE

Il file `.dockerignore` definisce i percorsi esclusi dal contesto inviato a Docker durante la costruzione dell'immagine.

La configurazione utilizzata è:

```
data/
__pycache__/
*.pyc
.venv/
.git/
```

Le esclusioni hanno le seguenti finalità:

Percorso	Motivazione
`data/`	Esclude il database operativo dalla build

`__pycache__/`	Esclude i file della cache Python
`*.pyc`	Esclude i bytecode Python
`.venv/`	Esclude l'ambiente virtuale locale
`.git/`	Esclude i metadati del repository Git

L'esclusione della directory `data` è particolarmente importante.

Il database contiene lo storico delle misurazioni e deve rimanere separato dall'immagine applicativa.

Questa scelta permette inoltre di evitare l'inclusione accidentale dei dati all'interno di un'immagine distribuita.

9.7 CONFIGURAZIONE DOCKER COMPOSE

Docker Compose permette di descrivere e gestire i servizi applicativi mediante un file YAML.

Nel progetto ChainSense2 viene utilizzato il file `compose.yaml`.

La configurazione completa è:

```
services:
  dashboard:
    build: .
    image: chainsense2-dashboard:1.1
    container_name: chainsense2-web

    ports:
      - "127.0.0.1:8501:8501"

    volumes:
      - ./data:/app/data:ro
```

Il file definisce un unico servizio denominato `dashboard`.

9.7.1 DEFINIZIONE DEL SERVIZIO

La direttiva `services` contiene l'elenco dei servizi gestiti da Compose.

Nel progetto è presente soltanto il servizio Streamlit.

Il processo di acquisizione seriale non viene gestito da Compose e deve essere avviato separatamente sull'host.

9.7.2 BUILD E IDENTIFICAZIONE DELL'IMMAGINE

```
build: .
image: chainsense2-dashboard:1.1
```

Il parametro `build: .` indica che il contesto di costruzione è la directory corrente del progetto.

Docker utilizza il Dockerfile presente in questa directory.

Il parametro `image` assegna all'immagine risultante il nome e il tag specificati.

Il tag `1.1` rappresenta un identificativo scelto per l'immagine. Non modifica automaticamente il numero di versione mostrato dall'applicazione Streamlit.

9.7.3 NOME DEL CONTAINER

```
container_name: chainsense2-web
```

La direttiva assegna un nome esplicito al container.

Questo permette di individuarlo facilmente attraverso i comandi Docker utilizzati per monitoraggio, diagnostica e arresto.

9.8 NETWORKING E PUBBLICAZIONE DELLA PORTA

Il file Compose configura la pubblicazione della porta attraverso la direttiva:

```
ports:
  - "127.0.0.1:8501:8501"
```

La sintassi è composta da:

```
IP_HOST:PORTA_HOST:PORTA_CONTAINER
```

I valori utilizzati hanno il seguente significato:

Elemento	Valore	Funzione
IP host	`127.0.0.1`	Limita il binding al loopback locale
Porta host	`8501`	Porta utilizzata dal browser Windows
Porta container	`8501`	Porta sulla quale ascolta Streamlit

Il server Streamlit ascolta sull'indirizzo `0.0.0.0` all'interno del container, mentre Docker pubblica il servizio soltanto sull'indirizzo di loopback dell'host.

Le due configurazioni operano su livelli differenti e non sono in contraddizione.

L'applicazione è raggiungibile dal computer host attraverso:

```
http://localhost:8501
```

La configurazione non è pensata per esporre direttamente il servizio ad altri dispositivi della rete locale o a Internet.

9.9 PERSISTENZA DEI DATI MEDIANTE BIND MOUNT

Uno degli aspetti principali dell'architettura Docker di ChainSense2 riguarda la gestione dello storico delle misurazioni.

I dati devono rimanere disponibili anche dopo l'arresto, la rimozione o la ricostruzione del container.

Per questo motivo il database SQLite viene conservato nella directory `data` del computer host.

Il collegamento viene definito nel file Compose:

```
volumes:
  - ./data:/app/data:ro
```

9.9.1 FUNZIONAMENTO DEL BIND MOUNT

La direttiva collega una directory del filesystem host a un percorso interno al container.

Elemento	Percorso
Directory host	`./data`
Directory container	`/app/data`
Modalità di accesso	`ro` — Read Only

La cartella `./data` contiene il database SQLite utilizzato dal processo Python.

All'interno del container, lo stesso file risulta accessibile attraverso:

`/app/data/chainsense.db`

Questo percorso è coerente con la logica utilizzata da `dashboard.py`, che individua il database relativamente alla posizione del proprio file sorgente.

9.9.2 SEPARAZIONE TRA LETTURA E SCRITTURA

Il processo Windows `serial_reader.py`, attraverso `storage.py`, dispone dell'accesso necessario per inserire nuove misurazioni.

Il container, invece, riceve la directory in sola lettura.

Questa configurazione riflette le responsabilità dell'architettura:

- Il processo host acquisisce e scrive i dati.
- Il container consulta il database e visualizza i risultati.

La modalità `ro` impedisce al container di modificare i file attraverso il bind mount.

Non impedisce invece al processo Windows di aggiornare il database originale.

9.9.3 CONSIDERAZIONI SULL'ACCESSO SQLITE

SQLite consente l'accesso al database da parte di più connessioni e gestisce la coerenza delle operazioni mediante meccanismi di locking e transazioni.

Nel prototipo, l'applicazione host scrive le misurazioni mentre la dashboard esegue interrogazioni di lettura.

Il funzionamento dipende anche dalle modalità di accesso al file e dal filesystem condiviso tra Windows e il container Linux.

La configurazione con mount in sola lettura richiede particolare attenzione nel caso di modifiche alla modalità journaling di SQLite, poiché alcune operazioni potrebbero richiedere l'accesso a file ausiliari.

Il deployment documentato mantiene il database sul filesystem host e non configura esplicitamente una modalità WAL.

La presenza del bind mount assicura la disponibilità del percorso, ma non sostituisce una strategia di backup o una verifica dell'integrità dei dati.

9.10 COSTRUZIONE DELL'IMMAGINE DOCKER

La costruzione dell'immagine può essere effettuata attraverso la Docker CLI.

Il comando utilizzato è:

```
docker build -t chainsense2-dashboard:1.1 .
```

Il parametro `-t` assegna un nome e un tag all'immagine.

Il punto finale `.` identifica la directory corrente come contesto di build.

Durante la costruzione, Docker esegue le istruzioni del Dockerfile nell'ordine previsto:

1. Recupera l'immagine base Python.
2. Imposta la directory di lavoro.
3. Copia il file delle dipendenze.
4. Installa le librerie richieste.
5. Copia il sorgente Streamlit.
6. Registra la configurazione della porta e del comando di avvio.

Al termine, l'immagine è disponibile tra quelle presenti localmente.

La verifica può essere eseguita con:

```
docker images
```

9.11 AVVIO MANUALE DEL CONTAINER

La dashboard può essere avviata direttamente mediante `docker run`, senza utilizzare Compose.

Nell'ambiente PowerShell il comando è:

```
docker run --rm -it --name chainsense2-web `
  -p 127.0.0.1:8501:8501 `
  --mount "type=bind,source=$($PWD.Path)\data,target=/app/data,readonly" `
  chainsense2-dashboard:1.1
```

I principali parametri sono:

Parametro	Funzione
`--rm`	Rimuove automaticamente il container al termine
`-it`	Mantiene la sessione interattiva con terminale
`--name`	Assegna un nome al container
`-p`	Pubblica la porta TCP
`--mount`	Collega la directory dei dati
`readonly`	Configura il mount in sola lettura

Il comando deve essere eseguito dalla directory del progetto contenente la cartella `data`.

Il carattere accent grave utilizzato nelle righe PowerShell permette di proseguire il comando sulla riga successiva.

9.12 DEPLOYMENT CON DOCKER COMPOSE

Il metodo principale previsto per gestire il servizio è Docker Compose.

La procedura viene eseguita dalla directory contenente `compose.yaml`.

9.12.1 COSTRUZIONE E AVVIO

```
docker compose up --build
```

Il comando costruisce l'immagine, se richiesto, e avvia il servizio definito nel file Compose.

L'opzione `--build` richiede la costruzione dell'immagine prima dell'avvio.

Per avviare il servizio senza mantenere il terminale collegato ai log:

```
docker compose up -d
```

In questa modalità il container viene eseguito in background rispetto alla sessione del terminale.

9.12.2 VERIFICA DEL SERVIZIO

Lo stato dei servizi può essere consultato mediante:

```
docker compose ps
```

Per visualizzare i messaggi generati dall'applicazione:

```
docker compose logs
```

Per seguire i log durante l'esecuzione:

```
docker compose logs -f
```

9.12.3 ARRESTO DEL SERVIZIO

Per arrestare e rimuovere i container gestiti dal progetto Compose:

```
docker compose down
```

Il comando non elimina automaticamente il database SQLite conservato nella directory host `data`.

Questa caratteristica permette di riavviare il servizio senza perdere lo storico delle misurazioni.

9.13 DIAGNOSTICA E VERIFICA DEL CONTAINER

Docker mette a disposizione diversi comandi per controllare lo stato dell'ambiente applicativo.

Comando	Utilizzo
`docker ps`	Elenca i container in esecuzione
`docker ps -a`	Elenca anche i container arrestati
`docker images`	Mostra le immagini locali
`docker logs chainsense2-web`	Visualizza i log del container
`docker inspect chainsense2-web`	Mostra la configurazione dettagliata
`docker exec -it chainsense2-web sh`	Apre una shell nel container attivo

9.13.1 VERIFICA DEI FILE NEL CONTAINER

Con il container avviato è possibile aprire una shell:

```
docker exec -it chainsense2-web sh
```

Successivamente è possibile controllare i percorsi:

```
pwd
ls /app
ls /app/data
```

La directory `/app` deve contenere il sorgente della dashboard e i file applicativi copiati dal Dockerfile.

La directory `/app/data` deve rendere visibile il database SQLite presente sull'host, a condizione che questo sia stato creato.

9.13.2 VERIFICA DEL COLLEGAMENTO AI DATI

La presenza del file `chainsense.db` all'interno del container conferma che il percorso è visibile.

La sola presenza del file non garantisce però che il database sia correttamente interrogabile.

Una verifica completa richiede anche il corretto caricamento delle misurazioni da parte di Streamlit.

In caso di database assente, la dashboard è predisposta per mostrare un messaggio informativo relativo alla mancanza di misurazioni.

9.14 AGGIORNAMENTO DELL'APPLICAZIONE

Quando viene modificato il codice di `dashboard.py`, l'immagine Docker già costruita non viene aggiornata automaticamente.

Il sorgente è infatti copiato nell'immagine durante la fase di build.

Per applicare una modifica è necessario ricostruire l'immagine e ricreare il container.

Con Docker Compose:

```
docker compose up -d --build
```

Il comando permette di aggiornare il servizio utilizzando il codice presente nel nuovo contesto di build.

Il database rimane sul filesystem host, separato dall'immagine applicativa.

9.14.1 VERSIONAMENTO DELLE IMMAGINI

Il nome `chainsense2-dashboard:1.1` identifica la versione dell'immagine configurata nel progetto.

In un processo di distribuzione più strutturato, a ogni versione rilasciata dovrebbe corrispondere un identificativo preciso, associato al codice sorgente e alle dipendenze utilizzate.

Nella configurazione corrente il tag è definito manualmente e non rappresenta un sistema automatico di versionamento.

9.15 LIMITI E CONSIDERAZIONI DI SICUREZZA

L'implementazione Docker è progettata per un ambiente locale di sviluppo e dimostrazione.

La configurazione presenta le seguenti caratteristiche e limitazioni:

- Il servizio Streamlit viene pubblicato soltanto sul loopback dell'host.
- Il database viene montato in sola lettura nel container.
- Il processo di acquisizione seriale rimane esterno a Docker.
- Non è configurato un sistema di autenticazione degli utenti della dashboard.

- Non sono previsti controlli di accesso specifici a livello applicativo.
- Le dipendenze Python non sono vincolate a versioni immutabili.
- Il container utilizza la configurazione utente prevista dall'immagine base, senza una direttiva `USER` dedicata nel Dockerfile.
- Non sono definiti healthcheck Docker personalizzati.
- Non è implementata una procedura automatica di backup del database.
- La configurazione non prevede il deployment su un server remoto.

Per un eventuale utilizzo in produzione sarebbero necessarie ulteriori verifiche su sicurezza, gestione delle versioni, protezione dei dati, continuità del servizio e accessibilità dell'interfaccia.

9.16 CONCLUSIONI

L'integrazione di Docker in ChainSense2 permette di distribuire ed eseguire la dashboard Streamlit all'interno di un ambiente Linux dedicato, mantenendo separato il processo di acquisizione seriale.

Il Dockerfile definisce il runtime applicativo, mentre Docker Compose permette di gestire in modo dichiarativo la costruzione, l'esecuzione, il networking e il collegamento al database.

L'utilizzo di un bind mount in sola lettura consente al container di accedere alle misurazioni archiviate sul computer host, senza includere il database nell'immagine Docker.

Questa architettura permette di aggiornare o ricreare il servizio di visualizzazione preservando lo storico delle misurazioni e senza richiedere modifiche al firmware ESP32.

La containerizzazione rappresenta quindi un ulteriore livello di separazione nell'architettura ChainSense2, migliorando l'organizzazione dell'ambiente di esecuzione e facilitando la riproducibilità del deployment.

10. INSTALLAZIONE, CONFIGURAZIONE E AVVIO DEL SISTEMA

10.1 INTRODUZIONE

Questo capitolo descrive la procedura necessaria per configurare e avviare ChainSense2 nell'ambiente di sviluppo utilizzato durante il progetto.

Il sistema comprende tre componenti da predisporre:

- Firmware ESP32, responsabile dell'acquisizione e dell'elaborazione delle Compact Frame.
- Applicazione Python, dedicata alla ricezione seriale e all'archiviazione SQLite.
- Dashboard Streamlit, eseguita all'interno di un container Docker.

La configurazione documentata prevede Windows come sistema operativo host e Docker Desktop con backend WSL 2.

10.2 PREREQUISITI

10.2.1 HARDWARE

Componente	Requisito
ESP32	DOIT DevKit V1
Chain2Gate Engine	Dispositivo configurato e funzionante

Display OLED	SH1106 128×64, interfaccia I²C
LED	Indicatori giallo e rosso
Computer	Windows con collegamento USB disponibile

I collegamenti tra i dispositivi devono rispettare la configurazione descritta nel Capitolo 3.

10.2.2 SOFTWARE

Software	Utilizzo
Visual Studio Code	Ambiente di sviluppo
PlatformIO	Compilazione e caricamento del firmware
Python 3	Esecuzione del lettore seriale
PySerial	Comunicazione Python–ESP32
Docker Desktop	Esecuzione dei container
WSL 2	Backend Linux di Docker Desktop

Le librerie Pandas e Streamlit vengono installate automaticamente durante la costruzione dell'immagine Docker.

10.3 PREPARAZIONE DEL FIRMWARE ESP32

Il firmware viene sviluppato e caricato mediante PlatformIO, utilizzando il framework Arduino.

Prima della compilazione è necessario verificare:

1. La selezione della scheda ESP32 DOIT DevKit V1.
2. La disponibilità delle librerie U8g2 e Wire.
3. La corretta configurazione dei collegamenti hardware.
4. La presenza del sorgente firmware aggiornato nel progetto PlatformIO.

Il caricamento avviene tramite la funzione **Upload** di PlatformIO.

Dopo il riavvio, il display OLED esegue la sequenza iniziale e passa alla schermata di attesa delle misurazioni.

La configurazione completa del file `platformio.ini` non è inclusa nei sorgenti di riferimento del presente manuale e deve essere verificata nell'ambiente PlatformIO utilizzato.

10.4 PREPARAZIONE DELL'AMBIENTE PYTHON

Il lettore seriale viene eseguito direttamente sul computer Windows.

È necessario verificare la disponibilità di Python e pip mediante PowerShell:

```
python --version
python -m pip --version
```

La libreria necessaria alla comunicazione seriale può essere installata con:

```
python -m pip install pyserial
```

Il file `requirements.txt` incluso nel progetto contiene le dipendenze della dashboard Docker e non comprende PySerial, utilizzato esclusivamente dall'applicazione host.

10.5 ORGANIZZAZIONE DEI FILE

La directory applicativa deve contenere almeno i seguenti elementi:

```
ChainSense2/
│
├── serial_reader.py
├── storage.py
├── dashboard.py
├── Dockerfile
├── compose.yaml
├── requirements.txt
├── .dockerignore
│
└── data/
    └── chainsense.db
```

La directory `data` e il database SQLite possono essere creati automaticamente dal modulo `storage.py` durante l'inizializzazione.

Il firmware ESP32 può essere conservato in un progetto PlatformIO separato.

10.6 CONFIGURAZIONE DELLA PORTA SERIALE

Nel file `serial_reader.py` sono definiti i parametri della comunicazione:

```
PORTA = "COM3"
VELOCITA = 115200
```

Prima dell'avvio, è necessario verificare in Gestione dispositivi di Windows quale porta COM è stata assegnata all'ESP32.

Se necessario, modificare il valore di `PORTA`.

La velocità deve corrispondere a quella configurata nel firmware.

Durante l'esecuzione di `serial_reader.py`, la porta non deve essere occupata contemporaneamente dal monitor seriale di PlatformIO o da altri programmi.

10.7 AVVIO DELL'ACQUISIZIONE DATI

Aprire PowerShell nella directory contenente i moduli Python ed eseguire:

```
python serial_reader.py
```

Lo script:

1. Inizializza il database SQLite.
2. Apre la porta seriale dell'ESP32.
3. Attende la ricezione dei messaggi JSON.
4. Estrae il tipo CF e la potenza.
5. Genera il timestamp.
6. Registra la misurazione nel database.

In presenza di un messaggio valido, il terminale visualizza le informazioni acquisite e la conferma del salvataggio.

Il processo deve rimanere in esecuzione per continuare a registrare nuove misurazioni.

10.8 AVVIO DELLA DASHBOARD CON DOCKER COMPOSE

Verificare che Docker Desktop sia avviato e che il motore Docker sia disponibile:

```
docker --version
docker info
```

Aprire PowerShell nella directory contenente `compose.yaml` ed eseguire:

```
docker compose up -d --build
```

Il comando costruisce l'immagine, crea il container e avvia il server Streamlit.

La configurazione Compose pubblica la porta 8501 sull'indirizzo di loopback del computer host.

La dashboard è accessibile dal browser all'indirizzo:

http://localhost:8501

Per verificare lo stato del servizio:

```
docker compose ps
```

Per consultare i messaggi del container:

```
docker compose logs -f
```

10.9 VERIFICA DEL FUNZIONAMENTO COMPLETO

Dopo aver avviato il firmware, il lettore Python e la dashboard, verificare i seguenti elementi.

Componente	Verifica
ESP32	Display inizializzato e firmware operativo
Chain2Gate	Ricezione delle notifiche CCP
Firmware	Riconoscimento CF3/CF21 e potenza
Python	Ricezione JSON e conferma del salvataggio
SQLite	Presenza dei record nella tabella `misurazioni`
Docker	Container `chainsense2-web` in esecuzione
Streamlit	Visualizzazione dei dati nel browser

Una verifica diretta del database può essere effettuata mediante una query SQL:

```
SELECT *
FROM misurazioni
ORDER BY id DESC
LIMIT 5;
```

La presenza di nuove righe conferma l'avvenuta registrazione delle misurazioni.

La dashboard deve successivamente visualizzare i dati disponibili, aggiornandoli secondo l'intervallo configurato.

10.10 ARRESTO E RIAVVIO

Per interrompere il processo di acquisizione Python, utilizzare `Ctrl+C` nel relativo terminale.

Per arrestare la dashboard:

```
docker compose down
```

Il comando arresta e rimuove il container gestito da Compose, senza eliminare il database conservato nella directory host.

Per riavviare la dashboard senza richiedere una nuova build:

```
docker compose up -d
```

Il lettore seriale deve essere riavviato separatamente, se precedentemente arrestato.

10.11 SEQUENZA OPERATIVA CONSIGLIATA

Per il normale utilizzo del sistema si raccomanda la seguente sequenza:

1. Collegare e alimentare i dispositivi hardware.
2. Verificare l'avvio del firmware ESP32.
3. Controllare la porta COM assegnata.
4. Avviare `serial_reader.py`.
5. Verificare la ricezione e il salvataggio delle misurazioni.
6. Avviare il servizio Docker Compose.
7. Aprire la dashboard nel browser.
8. Verificare l'aggiornamento dello storico.

La dashboard può essere avviata anche prima dell'arrivo di nuove misurazioni, purché il database sia disponibile.

10.12 CONCLUSIONI

La procedura di avvio di ChainSense2 mantiene indipendenti il firmware embedded, il processo di acquisizione Python e l'interfaccia web containerizzata.

La separazione permette di verificare individualmente ogni componente e di identificare più facilmente eventuali anomalie.

Il database SQLite costituisce il punto di collegamento tra acquisizione e visualizzazione, conservando le misurazioni indipendentemente dal ciclo di vita del container Docker.

11. DIAGNOSTICA, GESTIONE DEGLI ERRORI E TROUBLESHOOTING

11.1 INTRODUZIONE

ChainSense2 integra strumenti diagnostici distribuiti tra firmware ESP32, applicazione Python e ambiente Docker.

Questi strumenti permettono di verificare il funzionamento dei diversi componenti e individuare eventuali anomalie durante l'acquisizione, l'archiviazione e la visualizzazione delle misurazioni.

La diagnostica si basa principalmente su:

- Indicatori LED e display OLED.

- Messaggi e contatori del monitor seriale ESP32.
- Messaggi ed eccezioni dell'applicazione Python.
- Interrogazioni del database SQLite.
- Stato e log del container Docker.

La separazione dei componenti consente di localizzare i problemi verificando progressivamente il percorso dei dati.

11.2 DIAGNOSTICA DEL FIRMWARE ESP32

Il firmware dispone di contatori dedicati al monitoraggio della ricezione seriale.

Le informazioni vengono visualizzate attraverso la funzione `aggiornaSerialeMonitor()` dopo la ricezione di una notifica CCP valida.

Contatore	Significato
`trameValide`	Notifiche CCP riconosciute con checksum corretto
`trameIgnorate`	Trame integre non riconosciute come notifiche supportate
`erroriChecksum`	Messaggi con checksum errato
`erroriTimeout`	Ricezioni interrotte per timeout
`erroriOverflow`	Superamenti della capacità del buffer
`eventiTotali`	Somma dei contatori diagnostici

Il firmware utilizza inoltre due indicatori luminosi.

Il LED giallo, normalmente acceso, si spegne brevemente dopo la ricezione di una notifica valida.

Il LED rosso viene attivato per circa tre secondi in seguito a timeout, overflow o checksum errato su una notifica CCP riconoscibile.

11.2.1 PRINCIPALI ERRORI DI RICEZIONE

Anomalia	Possibile causa	Verifica
Nessuna trama ricevuta	Cablaggio UART, alimentazione o assenza di notifiche	Controllare RX/TX, GND e disponibilità dei dati
Errori checksum	Trama alterata o comunicazione non corretta	Controllare cablaggi e dati seriali
Timeout RX	Trama incompleta	Verificare collegamenti e continuità della ricezione
Overflow	Trama superiore alla capacità del buffer	Controllare lunghezza e struttura del messaggio
Trama ignorata	Codice o indirizzi CCP non supportati	Analizzare intestazione del frame

Il timeout locale del firmware è configurato a 300 ms, mentre il buffer di ricezione ha una capacità di 384 byte.

In seguito a timeout o overflow, la funzione `resetRicezione()` ripristina lo stato di attesa del delimitatore iniziale.

11.3 VERIFICA DELLA COMUNICAZIONE CCP

Per diagnosticare problemi tra Chain2Gate Engine ed ESP32 è necessario verificare la corretta struttura delle trame.

I principali elementi da controllare sono:

Campo	Valore atteso per le notifiche
STX	`0xF7`
Source	`0x7F`
Destination	`0x01`
MSG_CODE	`0x4C`
ETX	`0xFB`
Checksum	Somma dei byte dati modulo 256

Il firmware invia una risposta ACK completa quando riceve una notifica CCP riconoscibile con checksum corretto.

```
F7 01 7F 4C EF FB BB
```

In caso di checksum errato, su una notifica riconoscibile, viene trasmesso il NACK:

```
F7 01 7F 4C FF FB CB
```

11.3.1 RITRASMISSIONE DELLE NOTIFICHE

Durante lo sviluppo è stato osservato che alcune notifiche venivano trasmesse ripetutamente dal Chain2Gate Engine.

La verifica del protocollo ha evidenziato la necessità di inviare una trama di acknowledgment completa, comprensiva del codice messaggio CCP.

Dopo la correzione dell'ACK, le prove effettuate hanno mostrato notifiche con contatore di trasmissione progressivo e senza le precedenti ripetizioni identiche nei campioni esaminati.

In presenza di nuove duplicazioni è opportuno verificare:

1. Integrità della trama ricevuta.
2. Invio della risposta ACK sulla porta `Serial2`.
3. Correttezza del formato dell'acknowledgment.
4. Progressione del campo `TransCnt`.

La versione attuale non utilizza `TransCnt` per eliminare automaticamente eventuali notifiche duplicate.

11.4 DIAGNOSTICA DELLA COMUNICAZIONE ESP32–PYTHON

Il programma `serial_reader.py` riceve i messaggi JSON dalla porta seriale USB.

La configurazione di riferimento utilizza `COM3` a 115200 baud.

11.4.1 PROBLEMI COMUNI

Sintomo	Possibile causa	Intervento
Porta COM non disponibile	Porta errata o dispositivo scollegato	Verificare Gestione dispositivi
Accesso alla porta negato	Porta occupata da un altro programma	Chiudere il monitor seriale o l'applicazione concorrente

Nessun JSON ricevuto	ESP32 non trasmette dati riconosciuti	Verificare l'output del firmware
Errore decodifica JSON	Messaggio incompleto o malformato	Controllare il contenuto ricevuto
Errore nei dati ricevuti	Campo assente o valore non convertibile	Verificare `cf` e `potenza_w`

La funzione `main()` intercetta le principali eccezioni della libreria PySerial e della decodifica JSON.

In caso di errore della comunicazione seriale, lo script segnala l'anomalia ma non implementa una riconnessione automatica.

11.4.2 VERIFICA DEL MESSAGGIO JSON

Un messaggio valido deve rispettare la struttura prevista dal firmware.

```
{"cf":21,"potenza_w":2448}
```

Il lettore ignora le righe che non iniziano con `{`, permettendo di separare le misurazioni dalle informazioni diagnostiche.

La presenza di un JSON sintatticamente valido non garantisce automaticamente che il valore rappresenti una misurazione fisicamente attendibile.

11.5 DIAGNOSTICA DEL DATABASE SQLITE

Il database operativo è memorizzato nel file:

```
data/chainsense.db
```

Per verificare che le misurazioni vengano archiviate correttamente, è possibile interrogare la tabella `misurazioni`.

```
SELECT *
FROM misurazioni
ORDER BY id DESC
LIMIT 5;
```

La query restituisce le ultime cinque righe in ordine decrescente di identificativo.

11.5.1 VERIFICHE PRINCIPALI

Anomalia	Possibile causa	Verifica
Database assente	Inizializzazione non eseguita	Avviare e controllare `serial_reader.py`
Tabella assente	Schema non inizializzato	Verificare `inizializza_database()`
Nessun nuovo record	Mancata ricezione o errore di scrittura	Controllare terminale Python e query SQL
Database bloccato	Connessioni o transazioni concorrenti	Controllare i processi che accedono al file
Errore di apertura	Percorso o permessi non corretti	Verificare la directory `data`

Prima di modificare manualmente il database o cancellare record, è opportuno arrestare i processi coinvolti ed effettuare una copia di sicurezza.

11.6 DIAGNOSTICA DOCKER E STREAMLIT

La dashboard viene eseguita nel container `chainsense2-web`.

La verifica dello stato può essere effettuata mediante PowerShell:

```
docker compose ps
```

Per consultare i log:

```
docker compose logs -f
```

Per verificare le immagini disponibili:

```
docker images
```

11.6.1 PROBLEMI COMUNI DEL CONTAINER

Sintomo	Possibile causa	Intervento
Container non avviato	Docker Desktop non disponibile o errore di avvio	Verificare Docker e i log
Dashboard non raggiungibile	Container fermo o porta non pubblicata	Controllare Compose e porta 8501
Errore durante la build	Dipendenze o file non disponibili	Esaminare l'output di costruzione
Database non visibile	Bind mount errato	Verificare `./data:/app/data:ro`
Dashboard senza misurazioni	Database vuoto o non aggiornato	Controllare SQLite e lettore Python

11.6.2 VERIFICA DEI FILE NEL CONTAINER

Con il container in esecuzione:

```
docker exec -it chainsense2-web sh
```

All'interno della shell è possibile controllare il collegamento alla directory dei dati:

```
ls /app
ls /app/data
```

Il file `chainsense.db` deve risultare visibile nella directory `/app/data`, se precedentemente creato sull'host.

La presenza del file non costituisce da sola una verifica della corretta esecuzione delle query SQL.

11.7 DIAGNOSTICA DELLA DASHBOARD

La dashboard utilizza l'età dell'ultima misurazione per mostrare un'indicazione sullo stato dei dati.

Tempo dall'ultima acquisizione	Stato
Inferiore a 30 s	Misurazione recente
Da 30 a meno di 120 s	Misurazione non recente
Da 120 s in poi	Misurazione obsoleta

Questi indicatori non rappresentano una verifica diretta della connessione ESP32–Chain2Gate.

Una Compact Frame CF21 può non essere generata per un intervallo prolungato se non vengono rilevate variazioni sufficienti della potenza.

In presenza di una dashboard apparentemente non aggiornata è quindi necessario distinguere l'assenza di nuove misurazioni da un effettivo problema di comunicazione.

11.8 PROCEDURA DI TROUBLESHOOTING

Per individuare un'anomalia si consiglia di verificare il sistema seguendo il percorso dei dati, partendo dall'hardware.

Fase	Controllo
1	ESP32 acceso e firmware operativo
2	Ricezione delle notifiche CCP sul monitor seriale
3	Verifica checksum e decodifica CF3/CF21
4	Trasmissione del JSON sulla seriale USB
5	Acquisizione del messaggio da parte di Python
6	Inserimento della misurazione in SQLite
7	Accessibilità del database dal container
8	Aggiornamento della dashboard Streamlit

L'obiettivo è identificare l'ultimo componente che funziona correttamente e verificare quello immediatamente successivo.

Questa procedura permette di evitare modifiche contemporanee a più livelli del sistema, rendendo più semplice individuare la causa del problema.

11.9 LIMITI DELLA DIAGNOSTICA

La versione attuale di ChainSense2 fornisce strumenti di monitoraggio utili durante lo sviluppo e l'esecuzione locale.

Non sono tuttavia implementati:

- Un sistema centralizzato di raccolta dei log.
- La riconnessione automatica alla porta seriale.
- Un controllo continuo dello stato fisico del Chain2Gate.
- La rilevazione automatica delle notifiche duplicate.
- Un sistema di allarme remoto.
- Healthcheck applicativi dedicati al container Docker.

Questi elementi possono costituire possibili evoluzioni per un utilizzo più strutturato e continuativo.

11.10 CONCLUSIONI

La diagnostica di ChainSense2 è distribuita tra i diversi componenti del sistema e permette di controllare separatamente acquisizione, elaborazione, archiviazione e visualizzazione.

Il firmware fornisce contatori e segnalazioni locali, Python gestisce le principali eccezioni di acquisizione, SQLite permette verifiche dirette dei record e Docker mette a disposizione strumenti di monitoraggio del container.

La separazione delle responsabilità consente di affrontare le anomalie seguendo un percorso di verifica progressivo, semplificando le attività di manutenzione e debugging.

12. LIMITI, SVILUPPI FUTURI E CONCLUSIONI

12.1 STATO ATTUALE DEL PROGETTO

ChainSense2, nella versione documentata, costituisce un prototipo di monitoraggio energetico basato sull'integrazione di un sistema embedded con un'applicazione software di acquisizione, archiviazione e visualizzazione dei dati.

Il sistema è stato sviluppato secondo un'architettura modulare, nella quale ogni componente svolge una funzione specifica e comunica con gli altri attraverso interfacce definite.

Le funzionalità principali realizzate comprendono:

- Acquisizione e verifica delle notifiche CCP provenienti da Chain2Gate Engine.
- Decodifica delle Compact Frame CF3 e CF21.
- Estrazione della potenza elettrica istantanea.
- Visualizzazione locale mediante display OLED e indicatori LED.
- Trasmissione delle misurazioni in formato JSON.
- Acquisizione seriale tramite applicazione Python.
- Archiviazione persistente in un database SQLite.
- Consultazione dello storico mediante dashboard Streamlit.
- Esecuzione dell'interfaccia web all'interno di un container Docker.

Le diverse componenti sono state integrate e verificate durante le attività di sviluppo del prototipo.

12.2 LIMITI DELL'IMPLEMENTAZIONE

La versione attuale è stata progettata per un ambiente di sviluppo e dimostrazione, senza l'obiettivo di realizzare un sistema commerciale completo.

I principali limiti sono riassunti nella seguente tabella.

Componente	Limite attuale
Firmware ESP32	Decodifica limitata alle Compact Frame CF3 e CF21
Protocollo CCP	Implementazione parziale delle funzionalità disponibili
Acquisizione Python	Porta seriale configurata manualmente, senza riconnessione automatica
Database SQLite	Schema semplificato con un'unica tabella
Dashboard	Filtri temporali predefiniti e analisi limitata alla potenza istantanea
Docker	Containerizzazione della sola dashboard
Diagnostica	Assenza di un sistema centralizzato di monitoraggio e gestione degli allarmi

Il sistema non implementa il calcolo dell'energia consumata in kWh né una gestione completa di più dispositivi di misura.

La configurazione attuale è inoltre orientata all'esecuzione locale su un computer Windows.

12.3 POSSIBILI SVILUPPI FUTURI

L'architettura modulare permette di prevedere successive estensioni senza dover necessariamente riprogettare l'intero sistema.

Tra le principali evoluzioni possibili rientrano:

Estensione del firmware

Implementazione di ulteriori decoder per le Compact Frame supportate dal protocollo, con una gestione più completa dei campi informativi e delle verifiche di validità.

Miglioramento dell'acquisizione

Introduzione di una configurazione esterna della porta seriale, riconnessione automatica e controlli per individuare eventuali notifiche duplicate.

Evoluzione del database

Estensione dello schema SQLite per archiviare ulteriori grandezze elettriche, identificativi dei dispositivi e informazioni temporali provenienti direttamente dal contatore.

Ampliamento della dashboard

Introduzione di filtri avanzati, analisi di periodi temporali più estesi ed esportazione completa dello storico.

Distribuzione del sistema

Valutazione di un deployment su un dispositivo dedicato o su un server locale, con gestione automatizzata dell'avvio, dei log e dei backup.

Queste funzionalità rappresentano possibili sviluppi e non sono comprese nell'implementazione attuale.

12.4 CONSIDERAZIONI TECNICHE FINALI

La realizzazione di ChainSense2 dimostra la possibilità di integrare un dispositivo embedded con un sistema software per l'acquisizione e la consultazione delle misurazioni energetiche.

Uno degli aspetti principali dell'architettura è la separazione delle responsabilità tra i diversi livelli.

Il firmware ESP32 gestisce la comunicazione con il dispositivo energetico e interpreta le informazioni ricevute; l'applicazione Python acquisisce e registra le misurazioni; il database SQLite ne garantisce la conservazione; la dashboard Streamlit permette la consultazione attraverso un'interfaccia web.

La containerizzazione della dashboard introduce un'ulteriore separazione tra ambiente applicativo e sistema host, mantenendo il database indipendente dal ciclo di vita del container.

Queste scelte consentono di intervenire sui singoli componenti mantenendo interfacce di comunicazione definite.

12.5 CONCLUSIONE

ChainSense2 rappresenta un sistema prototipale completo nel proprio ambito funzionale, capace di acquisire informazioni da un contatore elettronico, elaborarle, archiviarle e renderle consultabili attraverso un'interfaccia web.

La soluzione integra tecnologie embedded, comunicazione seriale, programmazione Python, database relazionali e containerizzazione, organizzate secondo un'architettura modulare.

Pur mantenendo alcune limitazioni legate al carattere dimostrativo del progetto, la struttura sviluppata costituisce una base tecnica per successive estensioni e miglioramenti.

Il presente manuale documenta l'architettura, le componenti software, le modalità di configurazione e le principali procedure operative di ChainSense2 nella versione di riferimento.

ChainSense2 — Manuale tecnico e documentazione software

Versione documento: 1.0
Firmware ESP32: 1.0
Progetto sviluppato nell'ambito del tirocinio formativo presso Cinetix Group
Ottobre–novembre 2026